FreyaLab

Moscato ⊕ Merlot

1st Edition

Contents

Preface **4**

1 About FreyaLab **7**
I Electric Circuits . . . 10
II Digital . . . 10
III Hardware . . . 10
IV Information Systems and Science . . . 10
V Physical Technology and Science . . . 11
VI Bio-Electronics and Bio-Imaging . . . 11
VII Energy and Environment . . . 12

2 Build and Install FreyaLab **13**
I FreyaLab with TR-BDF2 VBIC . . . 15
i About DC, AC, Pulse Circuit . . . 18
ii Architectural Analysis . . . 19
iii DC Circuit Diagnostic Test . . . 35
iv AC Circuit Diagnostic Test . . . 35
v Pulse Circuit Diagnostic Test . . . 35
II FreyaLab with Backward Euler . . . 36
i DC Circuit Analysis with FreyaLab Using Backward Euler Integration Scheme 38
ii AC Circuit Analysis with FreyaLab Using Backward Euler Integration Scheme 44

3 Electric Circuits **47**
I DC Circuits . . . 48
II AC Circuits . . . 55
i Sinusoids and Phasors . . . 55
III Pulse Circuits . . . 56
IV Advanced Circuit Analysis . . . 56
i Introduction to the Laplace Transform . . . 56

4 Digital **57**

5 Hardware **59**

6 Information Systems and Science **61**

7 Physical Technology and Science **63**

8 Bio-Electronics and Bio-Imaging **65**

9 Energy and Environment **67**

10 Reference and Bibliography **69**

Preface

For Moscato, Solreya, Mithra, Catenary, Iyzumrae and Zefir.

For Malbec and Znane along with all the 8 Queens (Pinot Noir, Prosecco, Cava, Riesling, Port, Shiraz).

For Albert Silverberg, Camus and Miklotov who left TREVOCALL to guard me.

For Nature(Paris, Kala, Kathmandu, Big Tree, Cabernet Sauvignon, Aokigahara, Hoia Baciu, Jacob's Well, Mt Logan, etc) and my family Sauvignon Blanc: I have served, I will be of service.

For my human parents.

For my dogs who always accompany me working in Valhalla Projection, go to Puncak Bintang or Kathmandu: Sine Bam Bam, Kecil, Browni Bruncit, Sweden Sexy, Cambridge Klutukk, Milan keng-keng, Piano Bludut, Krem, Walnut and more will be adopted. To my cat who guard the home while I'm away with my dogs: London.

The one who moves a mountain begins by carrying away small stones - Confucius

Words have the power to both destroy and heal. When words are both true and kind, they can change our world - The Buddha

Meditation brings wisdom; lack of meditation leaves ignorance. Know well what leads you forward and what holds you back - The Buddha

A book to explain how we create a C++ library for electrical engineering from zero, started on May 31, 2026.



Figure 1: *Freya, thank you for everything, I could never have done it without you. I created this 3 days before Christmas in 2021*

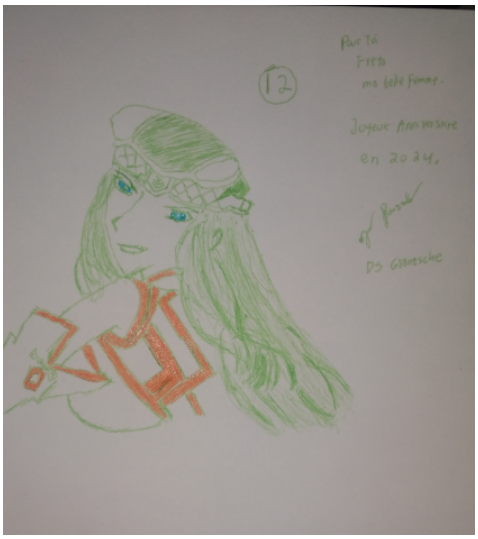


Figure 2: *Anniversary card for Freya in 2024.*

Chapter 1

About FreyaLab

On top of the mountain a heart shaped stone waiting for You, my eyes can't see, my body can't touch, but my heart knows it's You. Forever only You.
Find your life purpose and give your whole heart and soul to it - ***Buddha***

Electrical engineering is fun. With my first experience tinkering Arduino Uno from playing with LEDs to measure level of Carbon Monoxide nearby. I was reading on March 14th, 2023 about a brief history of Transistors, it is intriguing how a fact of "electrons form a barrier on the surface" could lead to the discovery of transistor. The fact could be discovered sooner if we are more sensitive, take a look more around us, learning from nature, we are all made out of atoms, protons, electrons. But, the most important thing of learning electrical engineering is, one day I want to learn how to make all magics in Harry Potter into reality, with Electrical Engineering. With "Wingardium Leviosa" I can easily make all trashes in the forest float and lead them into the end place for garbage.

This book is published for the very first time in github on September 26th, 2026 (The Harvest Moon). We are reading a comprehensive Electric Circuits book [2] and put a lot of the definitions, theorems, and formula here, we will also read and cite another books here to support our research, help our coding till FreyaLab become mature and probably can be used by a lot of people out there. We code the FreyaLab ourselves and make it for free as an open source for anyone who wants to learn electrical engineering and be an engineer, inventor, or scientist.

It is recommended that you have some knowledge in mathematics and physics before reading this book because we will use a lot of the formulas, definitions, and theorems from those two science branches. Several key mathematical branches and tools are essential for electric circuits and electrical engineering, they are:

1. **Complex Numbers and Phasor Analysis**
   Deals with numbers with a real and an imaginary component ($z = a + ib$). In Alternating Current (AC) circuits, voltage, and current waves have both a magnitude and a phase shift. Using complex numbers turns difficult trigonometric differential problems / time-varying AC calculations into simple algebraic operations, allowing engineers to represent a single complex impedance for resistors, inductors, and capacitors.

2. **Linear Algebra and Matrices**
   The study of vectors, vector spaces, and rectangular array of numbers. Large electrical

power grids and complex circuits with many loops and nodes generate massive systems of linear equations. Engineers use Matrix theory-specifically nodal and mesh analysis-to organize these equations into impedance and admittance matrices for fast computer simulation and load-flow studies. The use of algebra in circuits is to compute for unknown voltages, currents, resistances using fundamental relationships like Ohm's law ($V = IR$) and calculating series/parallel component values.

3. **Trigonometry**
   The study of the relationships between triangle sides and angles, and periodic wave functions ($\sin, \cos, \tan$). It is essential for Alternating Current (AC) theory as it helps to calculate phase angles, waveforms, impedance, and power factors in sinusoidal circuits.

4. **Laplace Transform**
   the Laplace transform converts time-domain differential equations into simple algebraic equations in the complex frequency (s) domain, making it easier to analyze linear time-invariant (LTI) electrical circuits, to evaluate control systems and transient responses. It converts calculus operations (such as differentiation and integration) into simple algebraic multiplication and division $\left(s, \frac{1}{s}\right)$. It handles initial conditions automatically, initial voltages across capacitors and initial currents through inductors are built directly into the s-domain circuit model rather than solved separately as arbitrary constants. It analyzes transients and ideal for studying sudden changes like switch toggles, step inputs, impulse responses, and second-order RLC circuit behaviors.

5. **Fourier Analysis**
   A mathematical technique that breaks down any complex, time-varying signal into a sum of simple sine and cosine waves. While the Laplace transform handles transients and general system stabiliy, the Fourier transform moves signals strictly into the frequency domain. It is vital for designing electronic filters, analyzing communication bandwidths, and processing audio or radio signals. Fourier transforms break complex or arbitrary signals down into individual frequency components for signal processing and filter design.

6. **Ordinary and Partial Differential Equations**
   Equations that relate a function to its own rates of change (derivatives). Components like capacitors and inductors store and release energy over time, meaning their behavior is naturally governed by differential equations, while PDEs are mandatory for understanding electromagnetic waves, transmission lines, and antenna fields (via Maxwell's equations). Differential equations can be used to model the transient and steady-state responses of second-order RLC circuits and dynamic systems.

7. **Calculus (Differential and Integral Calculus)**
   The mathematical study of continuous change and accumulation, it is useful in electric circuit since the fundamental circuit formulas rely on it. The voltage-current relationship for a capacitor is $v(t) = \frac{1}{c}\int i(t)\ dt$ and for an inductor is $v(t) = L\frac{di}{dt}$. Integration and differentiation are the baseline tools used to derive energy, power, and signal profiles.

8. **Probability Theory and Statistics**
   The mathematical framework for quantifying randomness, likelihood, and data distributions. Real world signals contain noise, and power systems face random component failures, load fluctuations, or weather disruptions. Probability is heavily used in reliability engineering, digital communications (to minimize bit errors), and statistical signal processing.

9. **Vector Calculus**
   The extension of calculus to vector fields, the differentiation and integration of vector fields (gradient, divergence, curl). It is essential for electromagnetic field theory, governing how electric and magnetic fields interact around wires, transformers, waveguides, and rotating machine coils. It is the foundational language of Maxwell's equations, used to design antennas, transmission lines, and analyze electromagnetic fields.

Several key physics branches and tools that are essential for electric circuits and electrical engineering:

1. **Classical Electromagnetism (Electrodynamics)**

   - **Maxwell's Equations:** Govern how electric and magnetic fields interact, change, and propagate. They explain how energy travels through cables, why high-frequency signals radiate, and how electromagnetic waves work.
   - **Faraday's Law of Induction:** States that a changing magnetic field induces an electromotive force (voltage) in a conductor. This principle is the basis for generators, transformers, inductors, and electric motors.
   - **Ampère's Law:** Describes how electric currents produce magnetic fields, making it essential for designing coils, electromagnets, and non-contact current sensors.

2. **Solid-State and Semiconductor Physics**

   - **Band Theory and Doping:** Explains energy bands in solids (valence and conduction bands) and how adding impurities (doping) creates p-type and n-type semiconductors.
   - **p-n Junctions and Transistors:** The behavior of charges across a p-n juction dictates how diodes, light-emitting diodes (LEDs), and transistors function. Transistors act as the fundamental building blocks for digital switches and analog amplifiers.

3. **Electrostatics**

   - **Electric Potential and Capacitance:** Covers how static or slow-moving charges store energy in electric fields.
   - **Capacitors:** Understanding charge storage and dielectric materials allows engineers to use capacitors for voltage smoothing, filtering noise, timing circuits, and power conditioning.
   - **Coulomb's Law and Electric Fields:** Help calculate electrostatic forces between charged particles, which is critical for preventing component breakdown from static discharge (ESD) and designing insulation.

4. **Current Electricity and Material Properties**

   - **Ohm's Law and Resistivity ($V = IR$):** Relates voltage, current, and resistance, governed at the atomic level by how free electrons scatter as they move through a crystal lattice.
   - **Temperature Dependence:** Knowledge of how thermal energy alters lattice vibrations and carrier mobility helps engineers manage component heating and thermal runaway.

5. **Thermodynamics and Electromechanical Energy Conversion**

   - **Joule Heating($P = I^2R$):** Quantifies how electrical energy converts into thermal energy when current passes through a resistive material, guiding heat-sink design and fuse ratings.

- **Conservation of Energy:** Ensures that power entering a network equals power consumed or stored across components, forming the foundation of Kirchoff's Circuit Laws.

Below is the outline for the chapters in this book.

I. Electric Circuits

This chapter will contain

1. DC Circuits
2. AC Circuits
3. Advanced Circuit Analysis

II. Digital

This chapter will contain

1. Digital Image Processing
2. Digital Signal Processing
3. Digital Systems Architecture

III. Hardware

This chapter will contain

1. Board Level Design
2. Introduction to VLSI Systems

IV. Information Systems and Science

This chapter will contain

1. Analog Communications Design Laboratory
2. Three-Dimensional Imaging (Design)
3. Principles of Sensing for Autonomy
4. Principles of Robot Autonomy
5. The Fourier Transform and Its Applications
6. Introduction to Stochastic Control with Applications
7. Signal Processing for Machine Learning
8. Information Theory
9. Introduction to Statistical Signal Processing
10. Feedback Control Design

V. Physical Technology and Science

This chapter will contain

1. Fundamentals of Analog Integrated Circuit Design
2. Semiconductor Devices for Energy and Electronics
3. Engineering Electromagnetics
4. Integrated Circuit Fabrication Processes
5. Digital MOS Integrated Circuits
6. Advanced Analog Integrated Circuit Design
7. Principles and Models of Semiconductor Devices
8. Applied Quantum Mechanics
9. Basic Physics for Solid State Electronics
10. Modern Optics
11. Guided Waves
12. Electromagnetic Waves
13. Introduction to Optical Fiber Communications

VI. Bio-Electronics and Bio-Imaging

This chapter will contain

1. Photonics
2. Neuroelectrical Engineering
3. Bioimaging
4. Biochips and Medical Imaging
5. Analytical Methods in Biotechnology
6. Neuroengineering Laboratory
7. Biodesign: Medical Technology Innovation

VII. Energy and Environment

This chapter will contain

1. Semiconductor Devices for Energy and Electronics
2. Power Electronics
3. Green Electronics
4. Electric Motors for Renewable Energy, Robotics, and Electric Vehicles
5. Introduction to Linear Dynamical Systems
6. Fundamentals of Energy Processes
7. Introduction to Sensing Networks
8. Energy Efficient Buildings
9. Introduction to Control Design Techniques
10. Quantum Mechanics of Nanoscale Materials
11. Electronic Materials Engineering
12. Solar Cells, Fuel Cells, and Batteries
13. Electronic and Photonic Materials and Devices Laboratory
14. Vehicle Dynamics and Control
15. Aerial Robot Design

Chapter 2

Build and Install FreyaLab

"Almost everything is written in C++, CERN uses it, NASA uses it, unless your name makes CERN and NASA obsolete, then you should learn C++ to be a better scientist and engineer"

We have created this C++ project on October 25th, 2026, after learning a lot about electric circuit theory and practice.

The story goes almost the same as SymIntegration that is branching out from SymbolicC++ version 3.35, we start from zero then expand further while also learning the theory, definitions, and formula of electrical engineering from a lot of textbooks. We believe that one day FreyaLab will be able to compute and simulate all kinds of electrical engineering problems. We will work hard, as it is not a walk in a park, probably one day we will be able to become a comprehensive, fast and efficient electrical engineering C++ library / software.

1. To download FreyaLab, open terminal / xterm then type:
   git clone https://github.com/glanzkaiser/FreyaLab.git

2. Enter the directory of the cloned FreyaLab repository then type:
   cd src
   make
   mv libfreyalab.so /usr/lib

   The Makefile will compile all the **.cpp** files in the **FreyaLab/src** directory.

```
CXX = g++

FREYALAB_DIR = ../src
LIB = libfreyalab.so
SOURCES = CircuitComponents.cpp
SOURCES += ACCircuit.cpp DCCircuit.cpp FileParser.cpp
    LogicGates.cpp math.cpp Netlist.cpp PulseCircuit.cpp

OBJS = $(addsuffix .o, $(basename $(notdir $(SOURCES))))
UNAME_S := $(shell uname -s)
```

```
CXXFLAGS = -fPIC
CXXFLAGS += -g -Wall -Wformat

## BUILD FLAGS PER PLATFORM

ifeq ($(UNAME_S), Linux) #LINUX
ECHO_MESSAGE = "FreyaLab from GFreya OS"

CFLAGS = $(CXXFLAGS)
endif

## BUILD RULES

%.o:$(SFREYALAB_DIR)/%.cpp
$(CXX) $(CXXFLAGS) -c -o $@ $<

all: $(LIB)
@echo Build complete for $(ECHO_MESSAGE)

$(LIB): $(OBJS)
$(CXX) -shared -o $@ $^ $(CXXFLAGS)

clean:
rm -f $(OBJS)
```

Code 1: *Makefile*

3. The dependencies:

- **C++ Compiler (GCC or Clang)**
- **OpenGL version 3.3**
  The open graphics library that we will use to visualize the electric circuit.
- **GLFW**

- **ImGUI v1.93**

- **ImPlot v1.1**

I. FreyaLab with TR-BDF2 VBIC

The main code in **https://github.com/glanzkaiser/FreyaLab/src** is a high-performance, second-order accurate Electronic Design Automation (EDA) circuit simulator written in standard C++. It is a mixed-signal SPICE circuit simulator that also has an interactive schematic CAD platform (with OpenGL 3.3, GLFW, ImGUi, Implot). It bridges the gap between raw scientific computing (advanced numerical linear algebra) and real-time visual hardware engineering by providing an all-in-one graphical desktop dashboard. Designed as an industrial-grade simulation core, it is structurally divided into decoupled infrastructure modules (such as parsing and sparse-matrix linear algebra sub-systems) and simulation execution engines (such as DC Operating Point, AC Transient, and high-speed Pulse engines). It uses a second-order TR-BDF2 (Trapezoidal Backward Differentiation Formula 2) composite integration engine, paired with a VBIC (Vertical Bipolar Intercompany Model) core topology mapping thermal and parasitic nodes without un-differentiable step discontinuities.

Physical & Mathematical Foundation
At the absolute center of FreyaLab its custom-tailored sparse numerical linear algebra core (**math.cpp**) we have:

1. **Gilbert-Peierls Sparse LU Factorization**
   Instead of relying on dense matrix operations that slow down heavily as circuits grow, FreyaLab implements a left-looking sparse column factorization using depth-first search (DFS) and topological sorting to compute symbolic and numeric matrix structures instantly.

2. **Compressed Sparse Row (CSR) Layout**
   Primitives stamp their physical mathematical footprints into ultra-compact, high-performance real and complex CSR matrix containers.

3. **SPICE Voltage Limiting (`pnjlim`)**
   To prevent exponential overflows during non-linear Newton-Raphson iterations, it features active junction voltage damping to stabilize non-linear device models.

4. **TR-BDF2 Discretization Architecture**
   TR-BDF2 splits every time step $\triangle t$ into two distinct segments. The first half-step uses the Trapezoidal rule to advance from t_n to a midpoint $t_{n+\gamma}$ (where $\gamma = 2 - \sqrt{2} \approx 0.585786$). The second half-step uses a second-order BDF method to advance from $t_{n+\gamma}$ to t_{n+1}. This achieves absolute stability (L-stability) to damp out high-frequency pulse switching ripples entirely while maintaining true second-order precision.
   Midpoint phase (Trapezoidal step $\gamma\triangle t$):

$$x_{n+\gamma} - \frac{\gamma\triangle t}{2} x_{n+\gamma} = x_n + \frac{\gamma\triangle t}{2} x_n$$

   Final phase (BDF2 step from $t_{n+\gamma}$ to t_{n+1}):

$$x_{n+1} - \frac{1-\gamma}{2-\gamma}\triangle t x_{n+1} = \frac{1}{\gamma(2-\gamma)} x_{n+\gamma} - \frac{(1-\gamma)^2}{\gamma(2-\gamma)} x_n$$

5. **The VBIC / Compact Non-Linear Formulation**
   The Vertical Bipolar Intercompany Model (VBIC) introduce a unified Kull-Nagel formulation to account for base current modulation (Early Effect) and high-injection knee roll-off smoothly.

It removes non-differentiable mathematical branches to guarantee perfectly smooth, quadratic Newton-Raphson convergence paths.

Physical & Mathematical Foundation
The simulator natively models standard analog and digital companion layers:

1. **Linear Primitives**
   Resistors, capacitors, inductors, and independent voltage/current sources.

2. **Non-Linear Devices**
   Junction diodes (quasi-static junction and carrier component models), MOSFETs (Shichman-Hodges models), and evaluated VBIC BJTs.

3. **Behavioral Digital Logic**
   Natively parses and evaluates fundamental logic gates (`AND`, `OR`, `NAND`, `NOR`, `XOR`, `XNOR`, `NOT`) directly alongside analog primitives within the same unified Modified Nodal Analysis (MNA) matrix space.

The Interactive Graphical Workspace Dashboard
The frontend user interface transforms FreyaLab from a command-line utility into a modern CAD tool using OpenGL, GLFW, GLAD, ImGui v1.93, and ImPlot v1.1:

1. **Live Netlist Compiler**
   An interactive text editor window allows engineers to alter netlists manually. Clicking compile immediately binds strings into matrix topology layouts and triggers the schematic compiler.

2. **Visual Schematic Canvas**
   A custom 2D grid viewport rendering engine complete with mouse scroll magnification zooming, pan-scrolling, and interactive selection. Components snap cleanly onto a 40px grid as you drag them around with the mouse.

3. **Real-Time Oscilloscope/Bode Plots**
   Powered by high-speed ImPlot buffers, the bottom of the interface acts as a multi-trace oscilloscope, rendering voltage traces with real-time interactive tracking crosshairs and log-scaled frequency gain graphs.

The FreyaLab simulation engine can absolutely be implemented as a foundational core for software tools used to design and manufacture tangible electronic products.
What we have built is a localized Electronic Design Automation (EDA) simulation core (similar to the kernel of commercial tools like Synopsys HSPICE, Cadence Spectre, or open-source ngspice). It does not directly manufacture physical hardware; rather it acts as the virtual proving ground where engineers test, stress, and optimize circuits before spending millions of dollars printing physical sillicon or assembling circuit boards.

In electrical engineering, SPICE stands for Simulation Program with Integrated Circuit Emphasis. It is the industry-standard software tool used to mathematically simulate and predict the behavior of electronic circuits before physically building them. Originally developed at UC Berkeley in the early 1970s, SPICE allows engineers to test their circuit designs in a virtual environment, saving millions of dollars in physical prototyping costs.
To run a simulation, SPICE relies on three foundational elements:

1. **The Netlist:** A text-based representation of the circit. It explicitly lists every component (resistors, capacitors, transistors) and describes how they are interconnected by specifying the connection points, called "nodes."

2. **Component Models:** Mathematical equations that define how individual parts behave. For example, a simple resistor follows Ohm's Law ($V = IR$), whereas a complex transistor model uses intricate semiconductor equations to dictate electron flow.

3. **The Solver Engine:** A powerful mathematicall engine that uses techniques like nodal analysis, linear algebra, and numerical integration to calculate voltages and currents at every point in the circuit over time.

Engineers use SPICE to run several crucial tests on a design:

1. **DC Analysis (Operating Point):** Calculates the steady-state voltages and currents in a circuit when a constant DC voltage is applied, determining how components are "biased."

2. **AC Analysis (Frequency Response):** Evaluates how the circuit responds to different signal frequencies. This is heavily used to design filters, audio equipment, and radio-frequency (RF) circuits.

3. **Transient Analysis:** Simulates the circuit's behavior over time in response to changing signals. It acts like a virtual oscilloscope, showing you exactly how waveforms distort, switch, or settle.

FreyaLab follows a modular, high-peformance architectue that strictly decouples the structural netlist parser, the sparse linear algebra solver backend, the time/frequency simulation orchestrators, and the OpenGL/ImGui interactive frontend visualization layer. The source tree is organized into four logical sub-systems (+1 for visualization):

1. **Core Infrastructure & Component Abstraction**
   These fules govern the raw electrical definitions and the base hardware interfaces for all network nodes.

   - **CircuitComponents.h:** The foundational blueprint for all simulated objects. Contains the abstract base class **Component** and handles the mapping of 1-based circuit netlist nodes to 0-based matrix states via **getSafeVoltage**. It declares all primitives including linear elements (**Resistor, Capacitor, Inductor**), non-linear physics components (**Diode, Mosfet, VbicBjtAdvanced**), independent inputs (**VoltageSource**), and mixed-signal components (**BehaviorLogicGateComponent**).
   - **LogicGates.h:** A lightweight static utility layer that provides behavioral evaluation logic for standard logic gates (**AND, OR, NOT**, etc) and handles binary nibble formatting for tracking data tables.

2. **Front-End Parsing & Circuit Modeling**
   This segment handles file management, string tokenization, and topological assembly.

   - **FileParser.h:** The bridge between textual schematic netlists and runtime data structures. It tokenizes standard SPICE-style strings, translates engineering metrix suffixes (e.g., **10k, 4.7u**), tracks label-to-node configurations using an internal lookup map, and inserts concrete objects into the netlist framework.

- **Netlist.h:** The central circuit topology container. Houses the complete collections of shared components and global sparse MNA arrays (**stateV, historicV, midpointV**). It drives the main simulation loop orchestrator and exposes non-const component inspectors for analysis tracking pipelines.

3. **Sparse Mathematical Algebra Backend**
   The heavy-lifting math core optimized for solving large, loosely coupled circuit matrices.

   - **math.h:** The numerical heart of the simulator. It implements real (**CsrMatrix**) and complex phasor-domain (**CsrMatrixComplex**) Compressed Sparse Row (CSR) data containers. It hosts the **CircuitMath** engine, featuring the Left-Looking Gilbert-Peierls sparse LU factorization matrix solver with automated partial pivoting, structural graph tracking (**gpDfs**), and SPICE-compliant junction voltage limiting loops (**pnjlim**).

4. **Specialized Simulation & Analysis Managers**
   These engine extensions implement the specific time-stepping and frequency sweep simulation rules.

   - **DCCircuit.h:** The static domain controller. Orchestrates the Newton-Raphson steady-state solver loop, generates the Operating Point (.OP) console matrix data, and computes Parametric sweeps (.STEP) across target elements to export raw CSV metric curves.
   - **ACCircuit.h:** The frequency-domain sweep (.AC) and time-domain time integration controller. Tracks adaptive TR-BDF2 time integration parameters and computes multi frequency linearized small-signal Bode matrix arrays.
   - **PulseCircuit.h:** A specialized high-speed transient (.TRAN) manager optimized for digital switching and sharp pulse transitions. It operates within precise timestep caps to accurately resolve high-frequency rise and fall edge topologies without suffering from matrix divergence.

5. **Interactive Visualization Canvas**
   The visual engine that gives FreyaLab its graphical interface.

   - **Visualization.h / VisualizationPointer.h:** The frontend workspace engine. Maps raw netlist topologies into physical model coordinates (**VisualComponent**). It coordinates viewport interactions (drag, zoom, orientation tracking), hosts vector data registers for immediate ImPlot waveform generation, and manages the dashboard UI widgets to let users toggle independent source stimulus parameters (DC, AC, or Pulse) on the fly.

i. About DC, AC, Pulse Circuit

The fundamental difference of DC, AC, and Pulse circuit lies in how the voltage and current behave over time. DC flows steadily in one direction, AC continuously reverses direction in a smooth wave, and pulse flows in one direction but turns on and off rapidly.

1. **Direct Current (DC) Circuits**
   In a DC circuit, electrons flow continuously in a single direction, moving from the negative terminal to the positive terminal.

   - **The Signal:** A steady, flat voltage line that does not change over time.
   - **Sources:** Batteries, solar panels, and USB chargers.
   - **Applications:** Electronic components (like microchips, displays, and sensors) require a stable, unpredictable energy level to process information without crashing.

Feature	Direct Current (DC)	Alternating Current (AC)	Pulse (Digital / PWM)
Direction of flow	Strictly unidirectional (one way)	Alternates (back and forth)	Unidirectional (one way)
Magnitude (Value)	Constant over time	Changes continuously (smooth wave)	Jumps abruptly between on/off states
Waveflow Shape	Flat, straight line	Sine wave (usually)	Square, rectangular, or spike
Primary Use Cases	Batteries, electronics, microchips	Home wall outlets, power grids	Data signals, motor control, LEDs

Table 2.1: *The Comparison of DC, AC, and Pulse*

2. **Alternating Current (AC) Circuits**
   In an AC circuit, the flow of electrons constantly reverses direction at a specific frequency (measured in Hertz, or cycles per second).

   - **The Signal:** A smooth, alternating wave (typically a sine wave) that crosses above and below zero volts.
   - **Sources:** Wall outlets and generators.
   - **Applications:** AC is highly efficient for transporting electricity over massive distances through power lines because it can be easily stepped up or down to high voltages using transformers.

3. **Pulse Circuits**
   A pulse circuit handles a hybrid form of current. It flows in only one direction (like DC) but turns on and off abruptly (like AC changes)

   - **The Signal:** A "chopped" wave (usually square or rectangular) that switches sharply between a high voltage level and zero volts.
   - **Sources:** Microcontrollers, computer clocks, and switching power regulators.
   - **Applications:** Pulse signals are the backbone of digital logic (where "On" is `1` and "Off" is `0`). They are also heavily used in Pulse Width Modulation (PWM) to precisely dim LEDs to control the speed of electric motors by changing how long the pulse stays "on" versus "off."

ii. Architectural Analysis

Here is the deep architectural analysis for the **Resistor** class

1. **Core Definitions**

   - **Conductance (g):** Defined as the reciprocal of resistance, $g = \frac{1}{R}$, measured in Siemens(S). In automated matrix circuit solvers, using conductance is preferred over resistance because parallel connections elegantly combine via simple matrix addition ($g_{total} = g_1 + g_2$).

- **Modified Nodal Analysis (MNA):** A generalized mathematical formulation used to construct systemic equations for circuit configurations. The engine splits the circuit into an internal matrix network equation:

$$G\boldsymbol{v} = \boldsymbol{b}$$

where G represents the structural network conductance matrix, $\boldsymbol{v}$ is the target node voltage state vector, and $\boldsymbol{b}$ constitutes independent stimulus source drives.
- **Sparsity Pattern:** The discrete collection of row/column matrix coordinates (i, j) where non-zero electrical values will reside. Isolating these topologies prior to execution ensures optimal allocation of Compressed Sparse Row (CSR) backends.

2. **Governing Electrical Theorems**

- **Kirchoff's Current Law (KCL):** KCL states that the algebraic sum of currents entering any node must equal zero

$$\sum I = 0$$

for a linear resistor tied between arbitrary network terminals **node1** (n_1) and **node2** (n_2), the branch current I_R escaping n_1 and entering n_2 is governed by:
Current leaving node 1: $I_{n_1} = g(V_{n_1} - V_{n_2})$
Current leaving node 1: $I_{n_2} = g(V_{n_2} - V_{n_1})$
This directly dictates the symmetrical $+g, -g$ mathematical patterns populated inside **stamp** and **stampAC**.
- **Ohm's Law:** The current traversing a two-terminal conductor is directly proportional to the potential difference drop across them:

$$I = \frac{V_{drop}}{R} = g \cdot (V_{n_1} - V_{n_2})$$

This rule forms the mathematical foundation behind the oprational fallback block executed inside the **calculateCurrent()** method.

3. **Concrete Mathematical Formulas and Stamping Rules**

- **The DC/Transient Linear Conductance Matrix Pattern:** When a standard structural element populates the real MNA sparse array G, it distributes values across its respective row and column indices based on 1-based netlist definitions mapped to 0-based matrix space $(n - 1)$:

$$\begin{bmatrix} \vdots & \dots & \vdots \\ +g & \dots & -g \\ \vdots & \dots & \vdots \\ -g & \dots & +g \\ \vdots & \dots & \vdots \end{bmatrix}$$

Self-conductance elements (matrix diagonal): Added at coordinates $(n_1 - 1, n_1 - 1)$ and $(n_2 - 1, n_2 - 1)$.
Mutual-conductance elements (matrix off-diagonal): Symmetrically assigned as negative terms at coordinates $(n_1 - 1, n_2 - 1)$ and $(n_2 - 1, n_1 - 1)$.

4. **Small-Signal Phasor Transform Domain Formula (stampAC):** For frequency domain AC sweeps, the circuit variables transform into complex numbers to represent magnitude and phase. For an ideal, pure resistor, the physical impedance holds zero imaginary reactive components ($Z_R = R + j_0$). Consequently, its complex small-signal conductance phasor calculation simplifies directly to:

$$Y_R(\omega) = g_{complex} = \frac{1}{R} + j_0$$

Because this expressions contains no frequency-dependent variables ($\omega = 2\pi f$), the element populates the complex system matrix A_c identically across all swept frequency ranges.

5. **Critical Division-by-Zero Protection Formula:** To prevent execution crashes or indefinite floating-point states (NaN) when processing uninitialized parameters or near-short-circuit traces, a numerical floor boundary is applied:

$$R_{safe} = \max(|R_{value}|, 10^{-20})$$

Class Method	Underlying Analytical Formula	Role in the Solver Pipeline
`stamp`	$G_{i,j} \leftarrow G_{i,j} \pm \left(\frac{1}{R_{safe}}\right)$	Populates the real conductance values into the static or transient solver matrix context G.
`stampAC`	$A_{c(i,j)} \leftarrow A_{c(i,j)} \pm \left(\frac{1}{R_{safe}} + j0\right)$	Mounts the complex numbers onto the frequency sween matrix A_c to evaluate magnitude and phase.
`getSparsityPattern`	Coordinate pairs: $\{(n_1 - 1, n_1 - 1), (n_2 - 1, n_2 - 1), \dots\}$	Preserves structural coordinate slots within the underlying Compressed Sparse Row arrays prior to simulation.
`calculateVoltageDrop`	$V_{drop} = V(n_1) - V(n_2)$	Subtracts node voltages (with built-in ground checking) to find the potential difference across the element.
`calculateCurrent`	$I_R = \frac{V_{drop}}{R_{safe}}$ or $I_R = \text{state}[ExtraIdx]$	Telemetry routine: prioritizes direct state lookups if an MNA row was allocated, otherwise applies Ohm's law.

Table 2.2: *Code execution trace and structural alignment for resistor*

Here is the deep architectural analysis for the **Diode** class

1. **Fundamental Definitions**

   - **Newton-Raphson Linearization:** A root-finding iteration framework used to solve non-linear algebraic systems. Because matrix equations like $G\boldsymbol{v} = \boldsymbol{b}$ require linear parameters, non-linear elements are mathematically replaced at each iteration by a linear companion model consisting of a dynamic conductance (g_d) in parallel with an equivalent independent current source (I_{eq}).
   - **Sparsity Allocation:** The predetermined configuration of non-zero structural locations within a matrix. In Compressed Sparse Row (CSR) sparse systems, memory allocation must happen before computation begins. The `getSparsityPattern` method acts as a topological scanner, securing the required coordinate blocks so that the solver knows where the diode's dynamic values will reside.
   - **Engineering Telemetry Probing:** The post-facto (after-the-fact) calculation of electrical parameters like voltage drops, branch currents, or total power dissipation once a Newton-Raphson iteration or time-step loop has successfully converged.
   - **Mathematical Consistency:** The requirement that state values calculated duing telemetry checks match the calculations used inside the matrix solver loops. For instance, the `calculateCurrent` method must incorporate the exact same non-linear damping parameters (V_{crit}) and stabilization features (G_{min}) used in the main matrix solver loop to prevent errors between the matrix state variables and the data logged to CSV files or UI plots.
   - **Thermal Voltage (V_{th}):** The foundational physical parameter linking charge carrier energy to temperature. At standard room temperature (300 K), $V_{th} = \frac{k_B T}{q} \approx 25.85$ mV.
   - **Critical Voltage (V_{crit}):** The mathematical boundary voltage where the rate of change of the diode's current matches numerical threshold limits. If a voltage update steps past V_{crit}, the exponential function begins to diverge rapidly or trigger floating-point overflows.
   - **Phasor Admittance (Y):** The complex frequency-domain expansion of electrical conductance. Calculated as $Y = G + j\omega C$, where G is the real small-signal conductance, $\omega = 2\pi f$ is the angular frequency, and C represents the aggregated dynamic charge storage capacitance.

2. **Governing Electrical Theorems, Physics Models & Structural Properties**
   Extended Modified Nodal Analysis (MNA) Design Rules
   In standard MNA formulation, circuit elements are categorized by whether they require an additional current tracking variable row in the system matrix:

   - **Passive-Non-Linear Voltage-Defined Elements:** Components like resistors, diodes, and MOSFETs express branch currents purely as a function of terminal node voltages ($I = f(V_{nodes})$). They map directly into the standard nodal matrix space without expanding the system size.
   - **Current-Defined Elements:** Components like voltage sources and inductors introduce an independent current variable, requiring an extra constraint row to resolve the mathematical system.

   Because the diode is voltage-defined, the parameter `int extraIdx` is explicitly cast to `(void)` to bypass additional matrix allocation.

Kirchoff's Voltage Law (KVL) Nodal Paradigm
KVL states that the directed sum of electrical potential differences around any closed loop is zero. In a nodal analysis solver, this implies that the voltage drop across any branch connecting an anode (n_1) to a cathode (n_2) is equal to the absolute difference tof their independent potentials relative to the reference ground:

$$V_d = V_{anode} - V_{cathode} = V(n_1) - V(n_2)$$

This principle forms the direct foundation behind the implementation of the `calculateVoltageDrop` utility function.

The Shockley Ideal Diode Equation
Governs the macroscopic, non-linear physical behavior of standard PN semiconductor junctions in steady-state operations:

$$I_d = I_s \left(e^{\frac{V_d}{nV_{th}}} - 1 \right)$$

where I_s is the reverse saturation current, V_d is the voltage drop across the junction, and n is the emission ideality factor (N).
Taylor Series First-Order Expansion (Companion Model Derivation)
To map the Shockley equation into linear MNA matrices, a first-order Taylor expansion is evaluated around the current iteration voltage guess V_{d0}:

$$I_d(V_d) \approx I_d(V_{d0}) + \frac{\partial I_d}{\partial V_d}|_{V_{d0}} (V_d - V_{d0})$$

By defining the dynamic small-signal conductance as $g_d = \frac{\partial I_d}{\partial V_d} = \frac{I_s}{nV_{th}} e^{\frac{V_{d0}}{nV_{th}}}$, this rewrites into a linear relationship:

$$I_d(V_d) \approx g_d \cdot V_d + (I_d(V_{d0}) - g_d \cdot V_{d0})$$
$$I_d = g_d \cdot V_d + I_{eq}$$

This directly yields the equivalent parallel current source formula deployed in step 4 of the code: $I_{eq} = I_d - g_d \cdot V_{d,lim}$.
Charge Storage and Small-Signal AC Dynamics
In high-frequency phasor domains, a physical diode exhibits dynamic charge modulation terms grouped into a dual capacitance architecture:

(a) **Diffusion Capacitance (C_{diff}):** Models injected minority charge carrier storage inside the junction during forward-bias conditions. It is directly proportional to current and conductance: $C_{diff} = \tau_t \cdot g_d$ (where τ_t is the transit carrier lifetime `tt`).

(b) **Depletion Capacitance (C_{dep}):** Models the changing space-charge width across the immovable ion lattice layer under varying voltage parameters.

3. **Concrete Mathematical Formulas Deployed in the Telemetry Pipeline**
Critical Voltage Boundary Formula
Determines the dynamic threshold before triggering voltage dampening or non-linear limiting loops:

$$V_{crit} = (nV_{th}) \cdot \ln \left(\frac{nV_{th}}{\sqrt{2} \cdot I_s} \right)$$

Exponential Overflow Clamping
Prevent numeric crashes by forcing an upper mathematical bound on the exponent:

$$\text{expTerm} = e^{\min\left(\frac{V_{d,lim}}{nV_{th}}, 40.0\right)}$$

The SPICE Standard Depletion Capacitance Model
To eliminate the asymptotic equation singularity (infinite value) that arises if a forward bias voltage approaches or equals the built-in junction potential ($V_d \to V_j$), the model splits into a piecewise equation around a designated forward boundary factor FC (typically set to 0.5):

$$C_{dep} = \begin{cases} C_{j0}\left(1 - \frac{V_{dc}}{V_j}\right)^{-M}, & \text{for } V_{dc} < FC \cdot V_j \\ C_{j0}(1 - FC)^{-M} + \frac{C_{j0} \cdot M(1-FC)^{-(M+1)}}{V_j}(V_{dc} - FC \cdot V_j), & \text{for } V_{dc} \geq FC \cdot V_j \end{cases}$$

This structure blends a classic reverse-bias depletion curve seamlessly into a continuous linear expansion overlay under highly forward-biased conditions.

The Spatial Sparsity Footprint Map
The coordinates generated by the diode mirror the structural configuration of a standard two-terminal component. For any non-ground node connection, the routine appends specific indexes to the sparse map container.

$$\text{Patterns Vector} \leftarrow \begin{cases} \text{Anode Self-Loop:}(n_1 - 1, n_1 - 1), & \text{if } n_1 > 0 \\ \text{Cathode Self-Loop:}(n_2 - 1, n_2 - 1), & \text{if } n_2 > 0 \\ \text{Mutual Coupling A:}(n_1 - 1, n_2 - 1), & \text{if } n_1, n_2 > 0 \\ \text{Mutual Coupling B:}(n_2 - 1, n_1 - 1), & \text{if } n_1, n_2 > 0 \end{cases}$$

Mathematical Soft-Voltage Limiting Function
During telemetry probing, if a solved state voltage spikes past the device's physical critical limits before final system convergence is reached, a localized soft-clamping mapping function handles the voltage vector parameter to prevent numerical overflow errors:

$$\text{If } V_d > V_{crit} \to V_{d,damp} = V_{crit} + \frac{V_d - V_{crit}}{1.0 + \frac{V_d - V_{crit}}{nV_{th}}}$$

This rational equation safely suppresses steep forward-voltage inputs, smoothly transitioning out of extreme exponential fields into a manageable linear growth curve as V_d spikes.

Extended Shockley Current Telemetry Model
To ensure the logged output matches the linear matrix solver computations exactly, the computed diode branch current includes the G_{min} stabilization conductance baseline:

$$I_{telemetry} = I_s \left(e^{\min\left(\frac{V_{d,damp}}{nV_{th}}, 40.0\right)} - 1.0 \right) + G_{min} \cdot V_d$$

4. **Mathematical Stamping & Topology Layout Matrix**
During transient iterations, the diode replaces its branch path with the dynamic companion

matrix components, stamping values into the global MNA sparse arrays at target slots $(n_1 - 1)$ and $(n_2 - 1)$:

$$G \leftarrow \begin{bmatrix} \vdots & \vdots \\ +g_d & -g_d \\ -g_d & +g_d \\ \vdots & \vdots \end{bmatrix}, \quad B \leftarrow \begin{bmatrix} \vdots \\ -I_{eq} \\ +I_{eq} \\ \vdots \end{bmatrix}$$

- G_{min} **Branch Injection Strategy:** The addition of `constexpr double gmin = 1e-12` injects a tiny parasitic resistor in parallel across the diode terminals. This provides a non-singular matrix guarantee, preserving connectivity if the diode drifts deep into cut-off mode ($g_d \to 0$).
- **The AC Phasor Stamp:** Small-signal sweeps are completely linear, meaning independent sources drive the network matrix equations while passive elements do not contribute vector values to B_c. Instead, the entire complex load populates the dynamic admittance matrix: $A_{c(i,j)} \leftarrow A_{c(i,j)} \pm (g_d + j\omega C_{total})$.

Class Method	Underlying Analytical Formula	Role in the Solver Pipeline
`stamp`	$g_d = \left(\frac{I_s}{nV_{th}}\right) e^{\frac{V_{d,lim}}{nV_{th}}} + G_{min}$, $I_{eq} = I_d - g_d \cdot V_{d,lim}$	Newton-Raphson loop: Linearizes the exponential junction characteristics. Stamps dynamic companion variables into the system matrices to evaluate the next solver guess. Vector target slots: matrix $G : \pm g_d$ and vector $\boldsymbol{b} : \mp I_{eq}$
`stampAC`	$Y = g_d + j\omega(C_{diff} + C_{dep})$	Small-signal sweep: Extracts linearized admittance parameters around the settled DC bias. Implements the multi-frequency phasor tracking response. Vector target slots: Matrix $A_c : \pm Y$.
`getSparsityPattern`	Matrix Coordinates: $\{\cup(n_i - 1, n_j - 1)\}$, for $n_i, n_j > 0$	Scans network topolofy and pre-allocates non-zero row/column blocks inside the Compressed Sparse Row (CSR) matrix during the initial netlist setup phase.
`calculateVoltageDrop`	$V_d = V(n_1) - V(n_2)$	Computes the potential difference between terminals while handling out-of-bounds nodes and the ground reference safely.
`calculateCurrent`	$I_{tot} = I_s \left(e^{\frac{V_{d,damp}}{nV_{th}}} - 1 \right) + G_{min} \cdot V_d$	Telemetry logging: Executes safe mathematical voltage dampening to calculate consistent branch currents for GUI scopes and CSV files. Post-processing telemetry tracking: Extracts accurate branch current data for the waveform visualization modules (`plotInputWaveForm`, `plotOutputWaveform`).

Table 2.3: *Code execution trace and structural alignment for diode*

Here is the deep architectural analysis for the **Capacitor** class

1. **Fundamental Definitions**
   In a mixed-signal Modified Nodal Analysis (MNA) circuit solver, the capacitor is a dynamic, linear, two-terminal passive energy storage device. It stores electrical energy within an internal electric field generated by structural polarization or displacement currents.
   Unlike static elements (such as resistors), its electrical state is fundamentally time-dependent. In the text-based SPICE configuration layer, it is instantiated via a standardized layout string tracking component definitions:

   ```
   C<name> <node_pos> <node_neg> <value>
   ```

   The component uses a 1-based index naming layout convention (0 or GND denotes the absolute global datum potential reference, constant at 0.0V) which is securely translated by the parsing layer down into internal 0-based matrix coordinate tracking registers.

2. **Governing Electrical Theorems, Physics Models, and Structural Properties**
   The physical behavior of a capacitor is governed by electro-static boundary models and standard displacement field variables:

   - **Constitutive Physical Equation (Charge-Voltage Domain):**

     $$q(t) = C \cdot v(t)$$

     where $q(t)$ is the instantaneous stored spatial charge in Coulombs (C), C is the physical structural capacitance rating in Farads (F), and $v(t) = v_{pos}(t) - v_{neg}(t)$ is the instantaneous terminal differential potential drop.
   - **The Continuity Equation & Core Differential Model:** The fundamental branch current $i(t)$ passing through the capacitor terminals is proportional to the time-rate-of-change of the electric displacement field:

     $$i(t) = \frac{dq(t)}{dt} = C \cdot \frac{dv(t)}{dt}$$

   - **Energy Conservation Limits:** The transient instantaneous potential energy $E_C(t)$ stored within the physical dielectric boundaries is modeled quadatically relative to voltage potentials:

     $$E_C(t) = \frac{1}{2} C \cdot [v(t)]^2$$

   - **Small-Signal Phasor Profile:** Under structural steady-state sinusoidal excitation at an angular tracking velocity of $\omega = 2\pi f$, the differential operator maps directly into the complex frequency plane ($s = j\omega$). This yields a standard linear complex admittance parameter:

     $$Y_c(\omega) = \frac{I(\omega)}{V(\omega)} = j\omega C$$

3. **Concrete Mathematical Formulas Deployed in the Telemetry Pipeline**
   To evaluate instantaneous telemetry characteristics without inserting extra rows or scaling matrix dimensions redundantly, the solver leverages discrete companion integration techniques. The pipeline deploys a composite TR-BDF2 multi-phase time integration strategy.

This method balances absolute stability margins against accuracy by splitting a standard temporal step $\triangle t$ into two sequential stages via an optimal fractional layout spacing constant:

$$\gamma = 2.0 - \sqrt{2.0} \approx 0.585786$$

Phase 0: DC Operating Point Analysis ($\triangle t \to 0$ or static initialization)
Under stable constant steady-state boundaries, $\frac{dv}{dt} = 0$. The ideal model scales to a structural open circuit. To eliminate floating nodes and prevent structural singularities inside the left-looking Gilbert Peierls linear algebra solver, a minimal numerical stabilization shunt leakage conductance (G_{min}) is introduced:

$$i_{DC} = G_{min} \cdot [v_{pos} - v_{neg}], \quad G_{min} = 10^{-12}\Omega^{-1}$$

Phase 1: Trapezoidal Rule Midpoint Substep ($t_n \to t_{n+\gamma}$)
An implicit, non-dissipative symmentric averaging step maps updates over the intermediate sub-interval $\triangle t_1 = \gamma \triangle t$:

$$q_{n+\gamma} - q_n = \frac{\gamma \triangle t}{2}(i_{n+\gamma} + i_n)$$

rearranging terms into an optimized Norton Companion Electrical Model ($i_{n+\gamma} = G_{eq1} \cdot v_{n+\gamma} + i_{eq1}$) isolates clear conductance and historical tracking components:

$$G_{eq1} = \frac{2C}{\gamma \triangle t}$$
$$i_{eq1} = -G_{eq1} \cdot v_n - \left(\frac{q_n}{\gamma \triangle t \cdot 0.5}\right)$$

where $q_n = C \cdot v_n$ tracks the converged absolute charge state from the previous timeline milestone.

Phase 2: BDF2 Endpoint Substep ($t_{n+\gamma} \to t_{n+1}$)
A second-order backward-difference tracking layer uses variable multi-step coefficients over the total timeline boundary step $\triangle t$, consuming both the historical t_n node points and the verified intermediate $t_{n+\gamma}$ midpoint elements:

$$i_{n+1} = \left(\frac{dq}{dt}\right)_{n+1} = \frac{1}{\triangle t}\left[\left(\frac{2-\gamma}{1-\gamma}\right) q_{n+1} - \left(\frac{1}{\gamma(1-\gamma)}\right) q_{n+\gamma} + \left(\frac{1-\gamma}{\gamma}\right) q_n\right]$$

mapping this equation onto the active Norton tracking layer ($i_{n+1} = G_{eq2} \cdot v_{n+1} + i_{eq2}$) isolates the following values:

$$c_1 = \frac{2-\gamma}{1-\gamma}, \quad c_2 = \frac{1}{\gamma(1-\gamma)}$$
$$G_{eq2} = c_1 \cdot \frac{C}{\triangle t}$$
$$i_{eq2} = -\frac{C}{\triangle t} \cdot [c_2 \cdot v_{n+\gamma} - (c_2 - c_1) \cdot v_n]$$

Multi-Frequency Small-Signal AC Frequency Sweep Analytics
When running frequency domain tracking scripts, the solver uses complex numbers ($\mathbb{C}$) across the structural admittance parameter space. To bypass pure DC division-by-zero crashes, it applies a defensive filter boundary:

$$Y_C(\omega) = \begin{cases} \sigma + j\omega C, & \text{if } |\omega| \geq 10^{-20} \\ G_{min} + j0, & \text{if } |\omega| < 10^{-20} \end{cases}$$

4. **Mathematical Stamping and Topology Layout Matrix**
   During matrix loading, the computed conductance parameters (G_{eq}) and historical current tracking variables (i_{eq}) are loaded into the global structural MNA matrix framework.
   Let the 1-based structural input nodes be n_1 and n_2. The solver maps these labels to internal row and column indexing registers using a 0-based translation protocol ($r_1 = n_1 - 1, r_2 = n_2 - 1$). If a node attaches to the global reference ground (label 0), its row/column matrix loading sequence is bypassed.
   The topological layout allocation modifies the Conductance Matrix G and the current right-hand side vector $\boldsymbol{b}$ as follows

$$\begin{bmatrix} +G_{eq} & -G_{eq} \\ -G_{eq} & +G_{eq} \end{bmatrix} \begin{bmatrix} -i_{eq} \\ +i_{eq} \end{bmatrix}$$

   Explicit algorithmic pattern mapping

   - **Cell Entry (row r_1, col r_1):** Accumulates $+G_{eq}$(current leaves node r_1 via its own self-conductance profile).
   - **Cell Entry (row r_2, col r_2):** Accumulates $+G_{eq}$ (current leaves node r_2 via its own self-conductance profile).
   - **Mutual Coupling Entry (row r_1, col r_2):** Accumulates $-G_{eq}$ (cross-conductance tracking potential interaction from node r_2).
   - **Mutual Coupling Entry (row r_2, col r_1):** Accumulates $-G_{eq}$ (cross-conductance tracking potential interaction from node r_1).
   - **Right-Hand Side Target Vector $\boldsymbol{b}[r_1]$:** Accumulates $-i_{eq}$ (injects matching equivalent Norton shortcut tracking elements into terminal node 1).
   - **Right-Hand Side Target Vector $\boldsymbol{b}[r_2]$:** Accumulates $+i_{eq}$ (injects matching equivalent Norton shortcut tracking elements into terminal node 2).

Class Method	Underlying Analytical Formula	Role in the Solver Pipeline
`stamp`	G updates via G_{eq}, $\boldsymbol{b}$ updates via $\mp i_{eq}$	Transient Matrix Assembly: Executes Newton-Raphson companion injection loops. Evaluates multi-phase TR-BDF2 time integration states and translates numerical continuous calculus values directly into linear matrix equations ($G \cdot \boldsymbol{v} = \boldsymbol{b}$) for the sparse solver.
`stampAC`	$Y_C(\omega) = G_{min} + j\omega C$	Phasor Matrix Assembly: Loads complex multi-frequency small-signal admittances into the specialized complex CSR framework ($A_c \cdot \boldsymbol{v}_c = \boldsymbol{b}_c$). Incorporates a defensive frequency-floor filter to shield the Gilbert-Peierls solver from zero-frequency division errors.
`getSparsityPattern`	Graph Edge Allocations: $\{(n_1 - 1, n_1 - 1), (n_2 - 1, n_2 - 1), (n_1 - 1, n_2 - 1), (n_2 - 1, n_1 - 1)\}$	Structural Initialization: Registers static structural memory non-zero allocation anchors during network topology analysis. Ensures that the Compressed Sparse Row (CSR) matrix pre-allocates memory for these cells before the math engine runs, eliminating runtime allocation overhead.
`calculateVoltageDrop`	$v_{drop}(t) = v_{n_1}(t) - v_{n_2}(t)$	State Telemetry Probing: Samples instant node absolute tracking voltages from the active solver state array. Resolves differential potential drops across terminals to feed calculation hooks, post-facto instrumentation monitors, and graphical UI plotting tracks.
`calculateCurrent`	DC: $i(t) = G_{min} \cdot v_{drop}(t)$, TR step: $i(t) = G_{eq1} \cdot v_{drop}(t) + i_{eq1}$, BDF2 Step: $i(t) = G_{eq2} \cdot v_{drop}(t) + i_{eq2}$	Banch Current Telemetry Extraction: Computes instantaneous physical current flows utilizing either active state-vector registers or historical companion values ($i = G_{eq} \cdot v + i_{eq}$). Vital for processing device power consumption and checking Local Truncation Error (LTE) metrics.

Table 2.4: *Code execution trace and structural alignment for capacitor*

Here is the deep architectural analysis for the **Inductor** class

1. **Fundamental Definitions**
   In a Modified Nodal Analysis (MNA) circuit solver, the inductor is a dynamic, linear, two-terminal passive energy storage device that stores energy within an internal magnetic field generated by branch current loops.
   Unlike voltage nodes or purely voltage-driven elements, an inductor imposes a strict branch relation constraint on the circuit topology. To capture this without numerical differentiation, the MNA convention treats the inductor's internal branch current (i_L) as an explicit, independent state variable added to the global solution vector.
   In a standard SPICE netlist string, it is declared using the following token layout:

   ```
   L<name> <node_pos> <node_neg> <value>
   ```

   The parser maps these 1-based text labels to automated integer markers, routing any ground references directly to Node 0 (absolute datum at 0.0 V), and pprovisions a dedicated 0-based tracking register called `assignedExtraRowIdx` for its internal branch line.

2. **Governing Electrical Theorems, Physics Models, and Structural Properties**
   The structural and physical properties of the inductor are characterized by magnetic flux linkages and inductive potential transformations:

   - **Constitutive Physical Equation (Flux-Current Domain):**

     $$\phi(t) = L \cdot i_L(t)$$

     where $\phi(t)$ represents the instantaneous linked magnetic flux in Webers (Wb), L is the core magnetic inductance rating in Henries (H), and $i_L(t)$ is the independent internal branch current passing from the positive terminal to the negative terminal.
   - **Faraday's Law of Induction & Core Differential Model:** The terminal potential drop $v_L(t)$ across the inductor coils is directly proportional to the time-rate-of-change of its magnetic flux path:

     $$v_L(t) = v_{pos}(t) - v_{neg}(t) = \frac{d\phi(t)}{dt} = L \cdot \frac{di_L(t)}{dt}$$

   - **Energy Storage Restraints:** The physical potential energy $E_L(t)$ bound within the active magnetic flux space maps quadratically relative to its instantaneous current profile:

     $$E_L(t) = \frac{1}{2} L \cdot [i_L(t)]^2$$

   - **Small-Signal Complex Impedance:** In the frequency domain under continuous sinusoidal steady-state stimulation at an angular velocity of $\omega = 2\pi f$, the differential tracking operator transforms into the complex plane ($s = j\omega$). This yields an absolute linear complex impedance model:

     $$z_L(\omega) = \frac{V(\omega)}{I_L(\omega)} = j\omega L$$

3. **Concrete Mathematical Formulas Deployed in the Telemetry Pipeline**
   To evaluate transient parameters over variable time windows without triggering numerical errors, the MNA solver applies a composite, multi-phase TR-BDF2 time integration strategy.

This divides each time boundary step $\triangle t$ into a Trapezoidal Rule midpoint segment and a Backward Differentiation Formula 2 (BDF2) tail phase using the optimal stability spacing metric:

$$\gamma = 2.0 - \sqrt{2.0} \approx 0.585786$$

Phase 0: DC Operating Point Analysis ($\triangle t \to 0$ or static initialization)
Under a constant static steady state, $\frac{di_L}{dt} = 0$, meaning the inductor collapses down to a zero-resistance ideal short circuit. The tracking constraints enforce:

$$v_{pos} - v_{neg} = 0.0V$$

To keep the global left-looking Gilbert-Peierls matrix solver safe from mathematical structural anomalies, the branch relationship column is set to zero-impedance limits.

Phase 1: Trapezoidal Rule Midpoint Substep ($t_n \to t_{n+\gamma}$)
An implicit integration block maps variables symmetrically across the midpoint spacing boundary $\triangle t_1 = \gamma \triangle t$:

$$i_{n+\gamma} - i_n = \frac{\gamma \triangle t}{2L}(v_{n+\gamma} + v_n)$$

rearranging this relationship to isolate voltage potentials yields the optimized Thévenin Companion Electrical Model ($v_{n+\gamma} - R_{eq1} \cdot i_{n+\gamma} = V_{th1}$) deployed in the runtime pipeline:

$$R_{eq1} = \frac{2L}{\gamma \triangle t}, \quad V_{th1} = v_n - R_{eq1} \cdot i_n$$

where i_n and v_n are the locked, converged current and voltage state drop parameters from the historical time coordinate milestone.

Phase 2: BDF2 Endpoint Substep ($t_{n+\gamma} \to t_{n+1}$)
The solver advances through the final step endpoint using a second-order backward multi-step approach. This step processes both the old t_n baseline values and the intermediate $t_{n+\gamma}$ midpoint results:

$$\begin{aligned} v_{n+1} &= L \cdot \left(\frac{di_L}{dt}\right)_{n+1} \\ &= \frac{L}{\triangle t}\left[\left(\frac{2-\gamma}{1-\gamma}\right) i_{n+1} - \left(\frac{1}{\gamma(1-\gamma)}\right) i_{n+\gamma} + \left(\frac{1-\gamma}{\gamma}\right) i_n\right] \end{aligned}$$

mapping this derivative onto the Thévenin linear format ($v_{n+1} - R_{eq2} \cdot i_{n+1} = V_{th2}$) extracts the active coefficients:

$$c_1 = \frac{2-\gamma}{1-\gamma}, \quad c_2 = \frac{1}{\gamma(1-\gamma)}$$

$$R_{eq2} = c_1 \cdot \frac{L}{\triangle t}$$

$$V_{th2} = -\frac{L}{\triangle t} \cdot [c_2 \cdot i_{n+\gamma} - (c_2 - c_1) \cdot i_n]$$

Multi-Frequency Small-Signal AC Frequency Sweep Analytics
For frequency sweep routines, the solver switches into complex algebra spaces ($\mathbb{C}$). To isolate

pure DC singular barriers, it injects a baseline stabilizer series resistance (R_{min}) into the phasor framework:

$$Z_L(\omega) = \begin{cases} 0.0 + j\omega L, & \text{if } |\omega| \geq 10^{-20} \\ R_{min} + j0, & \text{if } |\omega| < 10^{-20} \end{cases}$$

where $R_{min} = 10^{-12}\Omega$.

4. **Mathematical Stamping and Topology Layout Matrix**
During transient solver steps, the calculated Thévenin branch resistance values (R_{eq}) and dynamic history forcing functions (V_{th}) are loaded directly into the global MNA vectors.

Let n_1 be the positive node, n_2 the negative node, and let `idx` be the allocated internal auxiliary branch index. The input node coordinates are converted using 0-based tracking shifts ($r_1 = n_1 - 1, r_2 = n_2 - 1$), while ground references bypass matrix accumulation rows. The pre-allocated branch current row `idx` remains unshifted.

The topological structure updates the Conductance Matrix G and the right-hand side forcing vector $\boldsymbol{b}$ according to this structural blueprint:

$$\begin{bmatrix} 0 & 0 & +1.0 \\ 0 & 0 & -1.0 \\ +1.0 & -1.0 & -R_{eq} \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ V_{th} \end{bmatrix}$$

explicit algorithmic pattern mapping:

- **Cell Entry (Row r_1, Col idx):** Accumulates +1.0 (maps the incoming inductor branch current component exiting node r_1).
- **Cell Entry (Row idx, Col r_1):** Accumulates +1.0 (sparsity mirror link mapping positive node contributions onto the branch equation).
- **Cell Entry (Row r_2, Col idx):** Accumulates -1.0 (maps the outgoing inductor branch current component entering node r_2).
- **Cell Entry (Row idx, Col r_2):** Accumulates -1.0 (sparsity mirror link mapping negative node contributions onto the branch equation).
- **Branch Internal Impedance Entry (Row idx, Col idx):** Accumulates $-R_{eq}$ (loads the transient equivalent companion series resistance into the inductor's loop equation).
- **Right-Hand Side Target Vector $B[idx]$:** Accumulates V_{th} (loads the integrated multi-step memory forcing function into the independent loop condition row).
- **Right-Hand Side Target Vector $B[r_1]$ and $B[r_2]$:** Do not receive transient history components directly; current injections are driven implicitly through the column weights of state variable i_L.

Class Method	Underlying Analytical Formula	Role in the Solver Pipeline
`stamp`	G updates via ± 1.0 & $-R_{eq}$, $\boldsymbol{b}$ updates via V_{th}	Transient System Assembly: Executes Newton-Raphson companion integration stamps. Evaluates multi-phase TR-BDF2 time tracking steps to systematically load equivalent loop branch resistances (R_{eq}) and historical state forcing conditions (V_{th}) into the solver grid ($G\boldsymbol{v} = \boldsymbol{b}$).
`stampAC`	$Z_L(\omega) = R_{\min} + j\omega L$, $G[brRow][brRow] = -Z_L(\omega)$	Phasor Matrix Assembly: Maps the complex frequency-dependent series impedance profiles into the specialized multi-frequency CSR framework ($A_c\boldsymbol{v}_c = \boldsymbol{b}_c$). Employs an automated series baseline floor resistance ($R_{\min}$) to shield the solver from pure DC numerical division anomalies.
`getSparsityPattern`	Graph Edge Allocations: $\{(brRow, brRow), (n_1 - 1, brRow), (brRow, n_1 - 1), (n_2 - 1, brRow), (brRow, n_2 - 1)\}$	Auxiliary Stuctural Pre-allocation: Registers static structural memory coordinate connections between node potentials and the auxiliary branch current variable before execution. Ensures that the Compressed Sparse Row (CSR) matrix architecture completely avoids real-time structural allocation penalties.
`calculateVoltageDrop`	$v_{drop}(t) = v_{n_1}(t) - v_{n_2}(t)$	Differential State Probing: Samples instantaneous node absolute tracking voltages from the operational simulation state vector via `getSafeVoltage`. Resolves the differential electromotive potential drop across terminal bounds to drive calculation loops and plotting diagnostics.
`calculateCurrent`	DC: $i(t) = \text{state}[brRow]$, TR Fallback: $i(t) = i_{hist_n} + \frac{v_{drop}(t) - V_{th1}}{R_{eq1}}$, BDF2 Fallback: $i(t) = i_{eq_bdf} + \frac{v_{drop}(t)}{R_{eq2}}$	Banch Current Telemetry Extraction: Natively extracts the explicitly solved independent current tracking parameter from the core MNA state vector. Incorporates a robust analytical fallback to reconstruct accurate current parameters during post-facto instrumentation sweeps.

Table 2.5: *Code execution trace and structural alignment for inductor*

Here is the deep architectural analysis for the **MOSFET** class

1.

Here is the deep architectural analysis for the **VbicBJT** class

1.

iii. DC Circuit Diagnostic Test

iv. AC Circuit Diagnostic Test

v. Pulse Circuit Diagnostic Test

Architecture Summary of Responsibilities

This table outlines the separation of concerns between continuous numerical math engines, discrete multi-bit bus evaluators, and post-processing telemetry trackers within the SPICE solver.

Function / Method	Domain Location	Input Type	Primary Job
`std::tanh` (Continuous Math)	`Component::stamp()`	`double` (Voltages)	Ensures the **Newton-Raphson solver** does not crash at threshold boundaries during active MNA simulations.
evaluateBusGate	**main.cpp** & `Visualizer::executeActiveSimulation()`	Discrete States	Generates ideal, multi-bit **Truth Tables** and digital bus simulations rapidly without analog overhead.
`evaluateScalarGate`	Post-simulation probing logs	**double** (Voltages)	Converts finalized analog matrix voltages into clean `0` or `1` boolean printouts for human telemetry tracking.

Table 2.6: *Division of Labor Across Simulation Domains*

II. FreyaLab with Backward Euler

We have created this simulation engine for comparison and educational purpose only, it is located in **https://github.com/glanzkaiser/FreyaLab/Old Codes/FreyaLab with Backward Euler**, the main code in **https://github.com/glanzkaiser/FreyaLab/src** is a high-performance, second-order accurate Electronic Design Automation (EDA) circuit simulator written in standard C++.

The Component Architecture:

- **Core Structural Engine : CircuitComponents.h / CircuitComponents.cpp**
  This layer models the physics of individual electrical components. Instead of storing complex system layouts globally, each component knows how to independentally calculate its parameters and "stamp" its linear mathematical equivalent directly into the global MNA matrices (G and B).
  - **Transient Companion Layouts:** Linear elements (resistors) stamp values directly. Reactive elements (capacitors, inductors) utilize backward Euler companion networks (converting differential transformations into dynamic equivalent resistances and parallel historic current sources updated at each time step).
  - **Newton-Raphson Linearization:** Non-linear items (Diodes, BJTs) use the previous iteration's voltage values inside an internal relaxation loop to compute real-time dynamic conductance tangents and current intercepts.
- **Mathematical System Layer: math.h / math.cpp**
  This is the math processor for the application. It has zero knowledge of electrical concepts like nodes, resistors, or voltages; it only operates on dense rows and columns of linear systems ($A\boldsymbol{x} = \boldsymbol{x}$).
  - **LU Factorization with Partial Pivoting:** Factoring matrices using Doolittle/Crout sorting models. Row-swapping (pivoting) prevents divisions by zero, ensuring mathematical stability when evaluating near-singular states (such as uncharged capacitors or high-impedance nodes).
  - **Substitution Solvers:** Performs forward and backward substitutions on the factored system matrices to quickly calculate the next set of unknown circuit voltage states.
- **Frontend Utility Interface: FileParser.h / FileParser.cpp**
  This module acts as the translator between human-readable circuit text files and compiled machine structures.
  - **SPICE Parser:** Scans standard design netlists string token by string token to dynamically assign internal node numbers (mapping node **0** as the absolute system Ground).
  - **Scale Value Normalization:** Converts engineering suffix tokens (e.g., $1K$ to 1000.0, $10u$ to $10e-6$, $4.7e-3$ directly into accurate raw float representations).
- **Categorized Analysis Suite: DCCircuit, ACCircuit, and PulseCircuit**
  Rather than cluttering the solver routines with deep-nested loops, analytical tracking is broken down into separate domain execution wrapper packages:
  - **DCCircuit:** Assumes steady-state constraints, runs long-step relaxation matrix solves to isolate basic node parameters, prints the Operating Point Table, and loops source $V1$ value to write parametric sweep trends out to separate **.csv** data tracking metrics.
  - **ACCircuit:** Solves pseudo-transient alignments across quadrature intervals to extract phase shifts and complex magnitude trends. It logs operational behaviors across

log-scaled frequency inputs, printing targeted data and exporting data points out to **bode_plot_data.csv**

- **PulseCircuit:** Steps forward using fine time steps (nanoseconds) to compute time-domain performance maps, outputting structural metrics displaying reactive charging and discharging behaviors under square-wave pulse configurations.

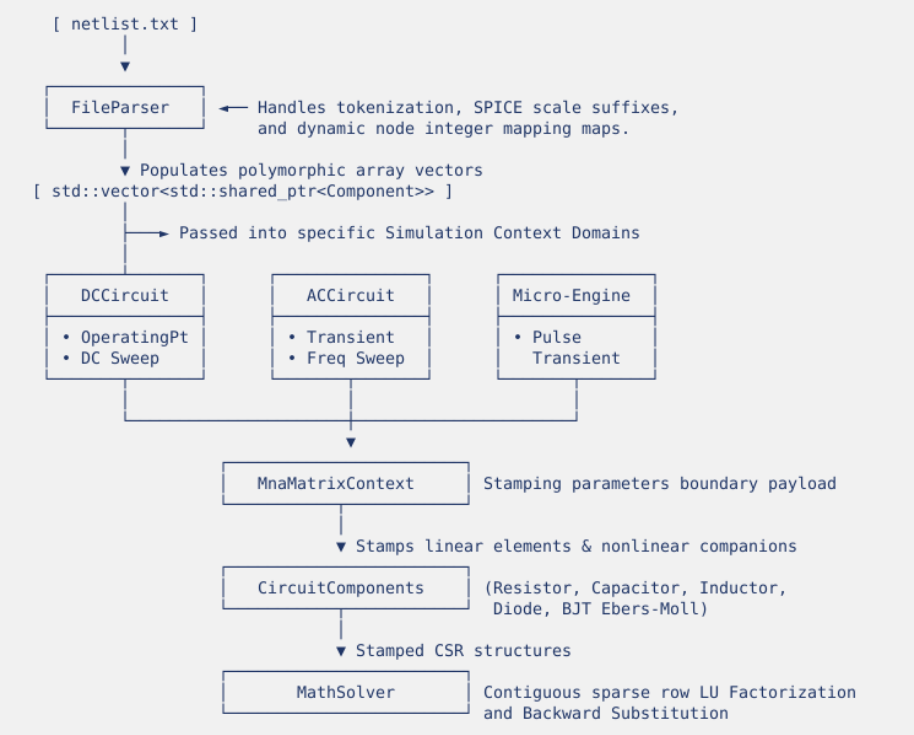


Figure 2.1: *The architecture for FreyaLab with Backward Euler separates matrix mechanics, netlist serialization, component modeling, and simulation control into decoupled functional layers. It implements SPICE-like circuit simulator engine using Modified Nodal Analysis (MNA), Newton-Raphson nonlinear integration, and Implicit Backward Euler transient integration.*

```
freyalab_backwardeuler/
├── include/
│   └── freyalab/
│       ├── CircuitComponents.h
│       ├── FileParser.h
│       ├── math.h
│       ├── DCCircuit.h
│       ├── ACCircuit.h
│       └── PulseCircuit.h
├── src/
│   ├── CircuitComponents.cpp
│   ├── FileParser.cpp
│   ├── math.cpp
│   ├── DCCircuit.cpp
│   ├── ACCircuit.cpp
│   ├── PulseCircuit.cpp
│   └── Makefile
└── main.cpp
```

Figure 2.2: *The directory structure.*

i. DC Circuit Analysis with FreyaLab Using Backward Euler Integration Scheme

We are using **FreyaLab** from **Freyalab with Backward Euler** as the starter / introduction, the most advanced one is using Trapezoidal method and its hybrid derivative and we will discuss it later on.

Suppose we have this **netlist.txt**:

```
V1 1 0 DC 17.0
R1 1 2 1k
R2 2 3 1k
R3 3 0 200
C1 2 0 10u
L1 2 3 4.7e-3
```

Code 2: *netlist.txt*

the voltage source is categorized as DC and we want to compute the exact electrical state of the circuit, the simulation engine uses Modified Nodal Analysis (MNA) copled with Compressed Sparse Row (CSR) matrix backend and LU Decomposition with partial pivoting.

1. **Mathematical Formulation: Modified Nodal Analysis (MNA)**
   Standard Nodal Analysis uses Kirchoff's Current Law ($\sum I = 0$) at each node. However, ideal voltage sources and inductors cannot be natively expressed as current functions of node voltages. MNA resolves this by introducing independent branch currents as explicit unknowns.
   The circuit is represented by the linear system

$$A\boldsymbol{x} = \boldsymbol{b}$$

$$\begin{bmatrix} G & B_{MNA} \\ B_{MNA}^T & D \end{bmatrix} \begin{bmatrix} \boldsymbol{v} \\ \boldsymbol{i}_{ext} \end{bmatrix} = \begin{bmatrix} \boldsymbol{i}_{independent} \\ \boldsymbol{v}_{independent} \end{bmatrix}$$

   where:

   - G is an $(N-1) \times (N-1)$ matrix tracking node conductances.
   - B_{MNA} and B_{MNA}^T map the connections of voltage sources and inductors.
   - D accounts for dependent control terms (all zeros for independent branches).
   - $\boldsymbol{v}$ is the vector of unknown node voltages (v_1, v_2, v_3). Node 0 is the ground reference (0 V).
   - $\boldsymbol{i}_{ext}$ contains the branch currents flowing through voltage source **V1** and inductor **L1**.
   - $\boldsymbol{i}_{independent}$ is the independent current injection vector.
   - $\boldsymbol{v}_{independent}$ contains independent voltage source values.

2. **Component Equations $ DC Boundary Theory**
   When **DCCircuit::solveMna** runs a static operating point analysis, it sets the time step parameters explicitly to $\triangle t = 0.0$. This tells the reactive elements to switch to their DC steady-state limits.

   Resistors (*R1, R2, R3*)
   Resistors follow Ohm's Law

$$I = G(V_A - V_B)$$

where $G = \frac{1}{R}$.

- **Stamp Rule:** Add $+G$ to positions (A-1,A-1) and $(B-1, B-1)$, and $-G$ to cross-terms $(A-1, B-1)$ and $(B-1, A-1)$ inside G.

Voltage Sources (*V1*)
An independent voltage source enforces a fixed potential drop: $V_{pos} - V_{neg} = V_{DC}$. It requires an extra row index k to solve for its unknown source current I_{V1}.

- **Stamp Rule:** Place the $+1.0$ at $(pos-1, k)$ and $(k, pos-1)$; place -1.0 at $(neg-1, k)$ and $(k, neg-1)$. Add the force voltage V_{DC} directly to the right-hand side vector position $B[k]$.

Capacitors (*C1*)
The continuous time-domain equation is $i(t) = C\frac{dv(t)}{dt}$ under Backward Euler discretization

$$i(t_{n+1}) = \frac{C}{\triangle t}(v(t_{n+1}) - v(t_n))$$

As $\triangle t \to 0$ for static DC operating points, the equivalent conductance $G_{eq} = \frac{C}{\triangle t} \to \infty$. To prevent matrix overflow and represent a steady-state condition where all field transients have decayed, an ideal capacitor collapses to a perfect Open Circuit ($G_{eq} = 0, I_{eq} = 0$). It injects nothing into the matrix.
Capacitor *C1* evaluates its static context ($dt = 0.0$), dropping to an Ideal Open Circuit, allowing no DC path to ground through that branch.

Inductors (*L1*)
The continuous equation is $v(t) = L\frac{di(t)}{dt}$. Discretized via Backward Euler:

$$v(t_{n+1}) = \frac{L}{\triangle t}(i(t_{n+1}) - i(t_n))$$

$$i(t_{n+1}) = \frac{\triangle t}{L}v(t_{n+1}) + i(t_n)$$

As $\triangle t \to 0$, the equation simplifies to $v(t_{n+1}) = 0$. This represents an Ideal Short Circuit. The inductor demands an extra matrix row allocation k_L to tract its stable internal through-current I_{L1}.

- **Stamp Rule:** Place the $+1.0$ at $(A-1, k_L)$ and $(k_L, A-1)$; place -1.0 at $(B-1, k_L)$ and $(k_L, B-1)$. The companion entry on the right-hand side remains 0.0.

Inductor *L1* flags an extra row requirement, stamping itself as an Ideal Short Circuit (0 V voltage drop source) directly tying Node 2 to Node 3.

Consequently, the circuit simplifies down to a linear parallel grid *R1* ($1k\Omega$), *R2* ($1k\Omega$) and *R3* (200Ω). The engine will immediately arrive at the correct quadratic solution in exactly 1 or 2 iterations.

3. **Assembling the Matrix Equation**
The circuit has 4 nodes (size 3 after dropping ground node 0) and 2 extra branch current requirements (*V1* and *L1*). This creates a total matrix dimension of 5×5.
Mapping the component locations from the netlist:

- *V1*: Positive Node 1, Negative Node 0 $\rightarrow$ allocates extra index row 4.
- *L1*: Node $A = 2$, Node $B = 3$ $\rightarrow$ allocates extra index row 5.

Applying the individual stamps constructs the following system matrix $A\boldsymbol{x} = \boldsymbol{b}$

$$\begin{bmatrix} (G_{R1}) & (-G_{R1}) & 0 & 1 & 0 \\ (-G_{R1}) & (G_{R1}+G_{R2}) & (-G_{R2}) & 0 & 1 \\ 0 & (-G_{R2}) & (G_{R2}+G_{R3}) & 0 & -1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & -1 & 0 & 0 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ i_{V1} \\ i_{L1} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 17.0 \\ 0 \end{bmatrix}$$

Substituting the exact component parameters ($G_{R1} = 0.001, G_{R2} = 0.001, G_{R3} = 0.005$):

$$\begin{bmatrix} 0.001 & -0.001 & 0.000 & 1.000 & 0.000 \\ -0.001 & 0.002 & -0.001 & 0.000 & 1.000 \\ 0.000 & -0.001 & 0.006 & 0.000 & -1.000 \\ 1.000 & 0.000 & 0.000 & 0.000 & 0.000 \\ 0.000 & 1.000 & -1.000 & 0.000 & 0.000 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ i_{V1} \\ i_{L1} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 17.0 \\ 0 \end{bmatrix}$$

4. **Linear Algebra Engine Algorithm**
   To solve the system above, the engine uses three steps defined in the **src/math.cpp** file.

   (a) **Step 1: CSR Allocation Strategy**
   MathSolver::finalizeStructure packs non-zero structural locations into three tight linear arrays (**values, colIndices, rowPointers**) to avoid storing empty elements.

   (b) **Step 2: Sparse LU Factorization with Partial Pivoting**
   The solver decomposes the matrix A into a lower triangular matrix L and an upper triangular matrix U, such that $\boldsymbol{p}A = LU$. The row permutation array $\boldsymbol{p}$ (the pivot vector) updates dynamically during runtime to move the largest numerical elements onto the main diagonal, preventing division-by-zero or numerical precision loss.

   For each column i from 0 to 4:

   i. Locate the row $r \geq i$ that has the maximum absolute value in the current column: $\max |A_{r,i}|$.
   ii. If this value is less than 10^{-20}, the matrix is mathematically singular and execution stops.
   iii. If $r \neq i$, swap row i with row r in the workspace array and record the swap in the **pivot** tracker.
   iv. For every row beneath the diagonal element ($r > i$), compute the scaling multiplier factor:
   $$L_{r,i} = \frac{A_{r,i}}{A_{i,i}}$$
   v. Subtract this scaled row to eliminate entries below the diagonal
   $$A_{r,c} \leftarrow A_{r,c} - L_{r,i} \cdot A_{i,c} \quad \forall c > i$$

   (c) **Step 3: Forward and Backward Substitution**
   Once factored, the system solves two simpler triangular systems using **MathSolver::sparseLUSolve**.

Forward Substitution ($Ly = pB$)

$$y_i = B_{pivot[i]} - \sum_{j=0}^{i-1} L_{i,j} \cdot y_j$$

Backward Substitution ($Ux = y$)

$$x_i = \frac{y_i - \sum_{j=i+1}^{4} U_{i,j} \cdot x_j}{U_{i,j}}$$

5. **Step-by-Step Analytical Solution**
We can trace this math by hand to verify the exact numbers from the simulation engine outputs

   (a) From row 4 of the system matrix

$$\begin{aligned} 1.0 \cdot v_1 &= 17.0 \\ v_1 &= 17.0\ V \end{aligned}$$

   (b) From row 5 (the inductor's short-circuit condition)

$$\begin{aligned} 1.0 \cdot v_2 - 1.0 \cdot v_3 &= 0 \\ v_2 &= v_3 \end{aligned}$$

   (c) Add row 2 and row 3 together to eliminate the internal inductor branch current variable i_{L1}

$$\begin{aligned} -0.001v_1 + 0.002v_2 - 0.001v_3 - 0.001v_2 + 0.006v_3 + i_{L1} - i_{L1} &= 0 \\ -0.001v_1 + 0.001v_2 + 0.005v_3 &= 0 \end{aligned}$$

   Substitute $v_1 = 17.0$ and $v_2 = v_3$ into this combined equation

$$\begin{aligned} -0.001(17.0) + 0.001v_3 + 0.005v_3 &= 0 \\ 0.006v_3 &= 0.0017 \\ v_3 &= \frac{17}{6} \\ v_3 &= 2.83333V \end{aligned}$$

   Since $v_2 = v_3$, thus $v_2 = 2.83333V$.

   (d) Solve for the component currents using the settled node voltages.
   Current through *R1*

$$I_{R1} = \frac{v_1 - v_2}{R1} = \frac{17.0 - 2.83333}{1000} = 0.0141667A$$

   Current through *R2*

$$I_{R2} = \frac{v_2 - v_3}{R2} = \frac{2.83333 - 2.83333}{1000} = 0.0000A$$

   Current through *R3*

$$I_{R3} = \frac{v_3 - 0}{R3} = \frac{2.83333}{200} = 0.0141667A$$

Source Current *V1* (i_{V1})
From row 1,

$$\begin{aligned} 0.001v_1 - 0.001v_2 + i_{V1} &= 0 \\ i_{V1} &= -I_{R1} \\ i_{V1} &= -0.0141667A \end{aligned}$$

The negative sign indicates current is leaving the positive terminal, matching SPICE passive sign convention.

Inductor Current *L1* (i_{L1})
By applying KCL at node 3 we will have

$$\begin{aligned} i_{L1} + I_{R2} &= I_{R3} \\ i_{L1} + 0.0 &= 0.0141667 \\ I_{L1} &= 0.0141667A \end{aligned}$$

FreyaLab simulation engine automatically calculates these exact values, prints them to the console window (xterm or terminal), and caches them for subsequent transient time steps.

Once the application finishes running, the **dcAnalyst.runDcSweepAnalysis(0.0, 12.0, 0.5);** command will save three **CSV** output files into the local current working directory containing data steps ranging from $0.0V$ to $12.0V$:

1. **dc_sweep_voltage.csv** (Tracks column headers for Node_1, Node_2, Node_3 values)

2. **dc_sweep_current.csv** (Tracks calculated branch current values across elements)

3. **dc_sweep_power.csv** (Tracks real-time thermal power losses and source consumption metrics)

Because this circuit is completely linear (it contains no diodes or BJTs to invoke iterative **pnjlim** or Newton-Raphson damping loops), the engine achieves exact numerical convergence in a single matrix step.
The actual execution timeline breaks down as follows:

- **File Parsing & Tokenization:** $\pm 0.15ms$ (Disk I/O string operations).
- **CSR Sparsity Allocation & Setup:** $\pm 0.04ms$ (Sorting row/column pairs via **std::set**).
- **Linear MNA Stamping:** $\pm 0.02ms$ (Evaluating companion representations with $dt = 0.0$).
- **Sparse LU Factorization & Substitution:** $\pm 0.03ms$ (Solving a minor 5×5 linear system framework).
- **Console I/O Rendering:** $\pm 0.50ms$ (Standard output synchronization bottlenecks).

Total computation time (excluding terminal printing): ± 0.24 milliseconds.

```
[Freyalab Simulator Core v2.0]
Loading circuit topology matrix from: netlist.txt...
Successfully parsed netlist schema structural elements.
Detected Node Array Size: 4 (including Ground Reference Node 0).
Total Active Physical Components Loaded: 6

Allocating network topology equations and finalizing CSR matrix sparsity patterns...
Executing Newton-Raphson Modified Nodal Analysis (MNA)...

========================================================================
                     DC OPERATING POINT TABLE
========================================================================
Name      Nodes          V-Drop(V)      Current(A)     Power(W)
------------------------------------------------------------------------
V1        1-0            17.0000        -0.0142        -0.2408
R1        1-2            14.1667        0.0142         0.2007
R2        2-3            0.0000         0.0000         0.0000
R3        3-0            2.8333         0.0142         0.0401
C1        2-0            2.8333         0.0000         0.0001
L1        2-3            0.0000         0.0142         0.0000
========================================================================

Simulation execution task completed successfully.
```

Figure 2.3: *A DC series circuit analysis with 4 nodes consist of three resistors of* $1,000\Omega$, $1,000\Omega$, *and* 200Ω, *respectively, one inductor of* 4.7×10^{-3}, *one capacitor of* 10^{-6} *Farad, and one voltage source of 17.0 Volts (**Freyalab with Backward Euler/Examples/Introduction DC/main.cpp**).*

ii. AC Circuit Analysis with FreyaLab Using Backward Euler Integration Scheme

We are using **FreyaLab** from **Freyalab with Backward Euler** for this example.

The **ACCircuit** module implements two core types of analysis for sinusoidal circuits: AC Time-Domain Transient Simulation and a Logarithmic Frequency Sweep (Bode Response Evaluation). Rather than relying on complex numbers ($a + bi$) which change the core matrix architecture, this solver handlaes everything using real-numbered Modified Nodal Analysis (MNA) equations. It captures phase shifts and magnitude properties by evaluating circuit state trajectories in the time domain.

1. **The Core MNA System Framework**
   At any given instance of frequency step, the circuit is solved using the standard matrix equation:

   $$G\boldsymbol{x} = \boldsymbol{b}$$

   where G (conductance / system matrix) is an $N \times N$ matrix combining nodal conductances and independent branch constraint coefficients, $\boldsymbol{x}$ (unknown vector) is a vector containing the unknown node voltages ($V_1, V_2, \dots$) followed by independent branch currents ($I_{V1}, I_{L1}, \dots$), and $\boldsymbol{b}$ (right-hand side vector) holds independent source values and historic equivalent current sources calculated from reactive components.

   For an AC or transient simulation, the nodal voltages and branch currents become continuous functions of time: $\boldsymbol{x}(t)$. The network behavior is governed by a system of continuous-time differential-algebraic equations (DAEs):

   $$G_c\boldsymbol{x}(t) + C_c\frac{\boldsymbol{x}(t)}{dt} = \boldsymbol{b}(t)$$

   where

   - G_c represets the static conductance memory allocations (resistors, independent source links).
   - C_c represents the memory states of reactive storage components (capacitors and inductors).
   - $\boldsymbol{b}(t)$ tracks time-dependent excitation vectors (e.g., $V_{in}(t) = V_p \sin(2\pi f t)$).

   To solve this on a digital computer, the engine slices continuous time into finite steps and numerically approximates the derivative $\frac{d\boldsymbol{x}(t)}{dt}$ at the current time point t_{n+1} using the value from the previous time point t_n.

2. **Time-Domain Companion Models (Backward Euler)**
   To evaluate reactive elements (capacitors and inductors) without differential equations, the solver approximates derivatives over a discrete time step ($\triangle t$) using the Backward Euler (Implicit Euler) integration scheme. It assumes the derivative at the next step is equal to the local forward slope evaluated strictly at the target point:

   $$\frac{dx}{dt}|_{t_{n+1}} \approx \frac{x(t_{n+1} - x(t_n))}{\triangle t}$$

   This transforms the differential equations into ordinary linear algebraic stamps composed of an equivalent conductance (G_{eq}) and an equivalent history current source (I_{eq}).

Capacitor Integration Formula
The fundamental physical constraint equation of a capacitor is

$$i_C(t) = C\frac{dv_C(t)}{dt}$$

applying the Backward Euler approximation at time t_n based on the previous step t_{n-1}

$$i_C(t_n) \approx C\frac{v_C(t_n) - v_C(t_{n-1})}{\triangle t}$$
$$i_C(t_n) \approx \left(\frac{C}{\triangle t}\right) v(t_n) - \left(\frac{C}{\triangle t}\right) v(t_{n-1})$$

rearranging this formula reveals a linear companion model (a parallel resistor and current source), this matches the equation of a resistor in parallel with an independent current source:

$$i_C(t_n) = G_{eq} \cdot v_C(t_n) - I_{eq}$$

- **Equivalent Conductance**
$$G_{eq} = \frac{C}{\triangle t}$$
stamped into matrix G.
- **Historic Current Intercept**
$$I_{eq} = \frac{C}{\triangle t} \cdot v_C(t_{n-1})$$
added to vector $\boldsymbol{b}$.

Inductor Integration Formula
The physical equation of an inductor is

$$v_L(t) = L\frac{di_L(t)}{dt}$$

approximating the current derivative over the discrete step $\triangle t$

$$v_L(t_n) \approx L\frac{i_L(t_n) - i_L(t_{n-1})}{\triangle t}$$

rearranging to fit MNA branch current requirements:

$$v_L(t_n) - \left(\frac{L}{\triangle t}\right) i_L(t_n) = -\left(\frac{L}{\triangle t}\right) i_L(t_{n-1})$$

- **Equivalent Resistance**
$$R_{eq} = \frac{L}{\triangle t}$$
stamped into the inductor's dedicated branch intersection slot in matrix G.
- **Historic Voltage Intercept**
$$V_{eq} = -R_{eq} \cdot i_L(t_{n-1})$$
added to vector $\boldsymbol{b}$ at the inductor's extra row index.

3. **Logarithmic Frequency Sweel & Polar Extraction Algorithm**
   For the frequency sweep (**runFrequencySweepAnalysis**), the solver calculates the steady-state AC magnitude and phase at specific frequencies without a full complex-number solver matrix. It does this using a Quadrature Sampling Method.
   The Quadrature Sampling Concept
   Any steady-state sinusoidal signal $f(t)$ at an angular frequency $\omega = 2\pi f$ can be written as a combination of a real (in-phase) component and an imaginary (quadrature) component

$$f(t) = A_{real}\cos(\omega t) + A_{imag}\sin(\omega t)$$

   The solver isolates these two components by capturing the circuit's response at two specific times within a single input period $T = \frac{1}{f}$

   (a) **Peak Sampling ($t_{peak} = \frac{T}{4} = \frac{1}{4f}$):** At this point, $\sin(\omega t) = 1$ and $\cos(\omega t) = 0$. The resulting state vector represents the pure real component (A_{real}).
   (b) **Quadrature Sampling ($t_{quad} = \frac{T}{2} = \frac{1}{2f}$):** At this point, $\sin(\omega t) = 0$ and $\cos(\omega t) = -1$. The resulting state vector isolates the imaginary component (A_{imag}).

   For every frequency step, the solver runs a two-step pseudo-transient simulation using a large step size $\left(\triangle t = \frac{1}{4f}\right)$ to directly calculate A_{real} and A_{imag} for every node voltage and branch current.

   Polar Conversion Calculations
   Once A_{real} and A_{imag} are extracted for a node or component, the solver uses polar equations to calculate the physical Magnitude and Phase

   - **Vector Magnitude** (A_{mag})

$$A_{mag} = \sqrt{(A_{real})^2 + (A_{imag})^2}$$

   - **Decibel Conversion (for Bode Magnitude)**

$$A_{dB} = 20 \cdot \log_{10}(A_{mag})$$

   - **Phase Angle (θ in degrees)**

$$\theta = \arctan 2(A_{imag}, A_{real}) \cdot \left(\frac{180.0}{\pi}\right)$$

Chapter 3

Electric Circuits

*"Everything on earth is depending on electric circuits currently, what do not depend are only Nature and True Runes." - **DS Glanzsche***

*"You are so beautiful." - **Joe Cocker***

If anyone recall Star Ocean 3, a PlayStation 2 game, then they probably will love electric circuits. It tells about how a spaceship can venture to another planet and galaxy, it is a game but it gives inspiration and challenge to be able to achieve that level of reality. On earth CE(Christ Era) 2023 the development of space technology depends a lot with basic knowledge of Electrical Engineering, Mechanical Engineering, Physics, and Mathematics. If we want to build a rocket, satellite or even a spaceship like Quark that is used in Star Ocean 3, we have to comprehend electric circuits, from building to maintaining, this knowledge is necessary, a lot of tedious works of remembering formulas, how to solve a circuit problems, analyzing a circuit, computing the response, and all that. In 1977, NASA' Voyager 1 was launched on a faster orbit and still functioning and sending back data. We might be able to capture the phenomena on outer space, and even capture the surface of Earth from the eye of the Voyager 1, it can be used to eliminate crime, reduce pollution, enforce better law and policies for better planet, as we human owe a lot to this planet that we have rent without really paying as much as they have given life to us since we first evolved or descended to this planet. The enormous potential excites us to learn more so one day we can build our own robots, engine, machine, rocket, spacecraft, spaceship, that are not only useful for science and engineering breakthrough, but reusable and won't add another space debris.

I. DC Circuits

- **Introduction**

[FL*] Electric circuit theory and electromagnetic theory are two fundamental theories upon which all branches of electrical engineering are built. Many branches of electrical engineering, such as power, electric machines, control, electronics, communications, and instrumentation, are based on electric circuit theory.

[FL*] An electric circuit is an interconnection of electrical elements. How a lamp can be turned on can be devised of simple electric circuit consisting of a battery, a lamp, and connecting wires.

- **Systems of Units**

[FL*] As electrical engineers, we must deal with measurable quantities, to determine the power needed to turn on a television, a radio, or the amount of gas needed to start a vehicle. It has different measurements, we will use an international measurement, the International System of Units (SI), so any professionals can understand when we are presenting our work in SI units.

Quantity	Basic unit	Symbol
Length	meter	m
Mass	kilogram	kg
Time	second	s
Electric current	ampere	A
Thermodynamic temperature	kelvin	K
Luminous intensity	candela	cd
Charge	coulomb	C

Table 3.1: *Six basic SI units and one derived unit relevant to this chapter.*

- **Charge and Current**

[FL*] The concept of electric charge is the underlying principle for explaining all electrical phenomena. Also, the most basic quantity in an electric circuit is the electric charge. **Charge** is an electrical property of the atomic particles of which matter consists, measured in coulombs (C).
We know from elementary physics that all matter is made of fundamental building blocks known as atoms and that each atom consists of electrons, protons, and neutrons. We also know that the charge e on an electron is negative and equal in magnitude to 1.602×10^{-19} C, while a proton carries a positive charge of the same magnitude as the electron.

Multiplier	Prefix	Symbol
10^{18}	exa	E
10^{15}	peta	P
10^{12}	tera	T
10^{9}	giga	G
10^{6}	mega	M
10^{3}	kilo	k
10^{2}	hecto	h
10	deka	da
10^{-1}	deci	d
10^{-2}	centi	c
10^{-3}	milli	m
10^{-6}	micro	μ
10^{-9}	nano	n
10^{-12}	pico	p
10^{-15}	femto	f
10^{-18}	atto	a

Table 3.2: *The SI prefixes.*

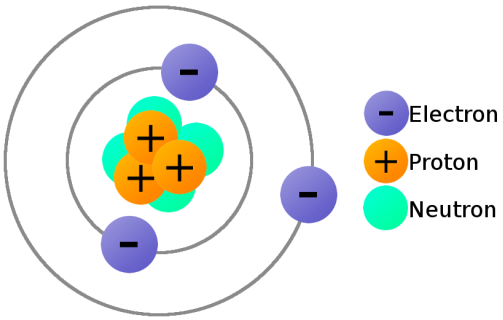


Figure 3.1: *An illustration of an atom, as the basic building block of all matter, that contains protons (have positive charge) and neutrons (have zero charge) inside the center nucleus, while electrons (have negative charge) are moving around the outside of the nucleus. Electrons have extremely small and nearly zero mass / -1.67×10^{-27} kg compared to protons that has about 1 atomic mass unit / -9.109×10^{-31} kg (similar to neutrons). The negative charge of the electrons and the positive charge of the protons attract each other, which keeps the electrons bound around the nucleus. An ordinary atom has an equal number of protons and electrons, their opposite charges cancel out, making the overal charge of the atom neutral. If an atom gains or loses electrons, it becomes a charged particle called an ion.*

Some important things about electric charge:

1. The coulomb is a large unit for charges. In 1 C of charge, there are $\frac{1}{1.602\times10^{-19}} = 6.24 \times 10^{18}$ electrons. Thus realistic or laboratory values of charges are on the order of $pC, nC, \mu C$.
2. According to the experimental observations, the only charges that occur in nature are integral multiples of the electronic charge $e = 1.602 \times 10^{-19}$ C.
3. The law of conservation of charge states that charge can neither be created nor destroyed, only transferred. Thus, the algebraic sum of the electric charges in a system does not change.

In DC circuits, we primarily track the movement of negative charge carriers (electrons).

[FL*] We now consider the flow of electric charges. A unique features of electric charge of electricity is the fact that it is mobile; that is, it can be tansferred from one place to another, where it can be converted to another form of energy.
When a conducting wire (consisting of several atoms) is connected to a battery (a source of electromotive force), the charges are compelled to move; positive charges move in one direction while negative charges move in the opposite direction. This motion of charges creates electric current. It is conventional to take the current flow as the movement of positive charges. This convention was introduced by Benjamin Franklin (1706-1790), the American scientist and inventor.

Electric Current is the time rate of change of net charge flowing through a specified cross-sectional area, measured in amperes (A). Mathematically, the relationship between current i, charge q, and time t is

$$i \triangleq \frac{dq}{dt} \tag{3.1}$$

where current is measured in amperes (A), defined as 1 Coulomb per second (C/s),

$$1\ ampere = 1\ coulomb/second$$

The charge transfered between time t_0 and t is obtained by integrating both sides of Eq. (3.1). Thus, we obtain

$$Q \triangleq \int_{t_0}^{t} i \, dt \tag{3.2}$$

The way we define current as i in Eq. (3.1) suggests that current need not be a constant-valued function.

[FL*] For a constant accumulation the formula to compute charge is

$$Q = I \cdot t$$

Thus, for steady-state Direct Current (DC), where the flow rate is constant the formula to compute the charge is

$$I = \frac{Q}{t} \tag{3.3}$$

[FL*] There are different ways of looking at direct current and alternating current. The best definition is that there are two ways that current can flow: It can always flow in the same direction, where it does not reverse direction, in which case we have direct current (DC). These currents can be constant or time varying. If the current flows in both directions, then we have alternating current (AC).
A **direct current** (DC) flows only in one direction and can be constant or time varying. An **alternating current** (AC) is a current that changes direction with respect to time.

Once we define current as the movement of charge, we expect current to have an associated direction of flow. Based on this convention, a current of 10 A may be represented positively or negatively depends on its direction.

[FL*] A current reading or value appears negative in a direct current (DC) circuit when the actual flow of charge moves in the opposite direction of the chosen reference or measurement setup.

1. **The Direction Rule**
   Standard engineering uses conventional current, which assumes current flows from the positive (+) terminal to the negative (-) terminal.
2. **The Reality**
   Actual electrons carry a negative charge and physically flow from the negative (-) terminal to the positive (+) terminal.
3. **The Sign Flip**
   If the meter or math tracks real electron movement or faces an inverted reference frame, the resulting value shows up with a negative sign (-).

Common reasons for a negative current reading:

1. **Reversed Multimeter Leads**
   Connecting a digital multimeter's red (positive) lead to the negative side of the circuit and the black (negative) lead to the positive side results in a negative current value on the screen.
2. **Component Orientation**
   Installing a polarized component (like an ammeter, diode, or DC power monito) backward relative to the circuit's actual flow direction forces the meter to read a negative value.
3. **Circuits Analysis Assumptions**
   During mathematical circuit analysis (using Kirchoff's Current Law), guessing a

reference arrow for current in the wrong direction yields a negative number for the final calculated current value, meaning the current flows opposite to the initial guess.

When we analyze a DC circuit using math (like Kirchoff's laws or nodal analysis), we must first guess or assign an arbitrary reference direction for the current through a component.

- **Positive Current ($I > 0$):** The actual charge is flowing in the exact direction you assumed or drew on your diagram.
- **Negative Current ($I < 0$):** The numerical value turns out negative because the real charge is flowing in the opposite direction of the chosen arrow. It does not mean the circuit is broken; it just means the current goes the other way.

- **Voltage**

[FL*] To move the electron in a conductor in a particular direction requires some work or energy transfer. This work is performed by an external electromotive force (emf). The emf is also known as voltage or potential difference. The voltage v_{ab} between two points a and b in an electric circuit is the energy (or work) needed to move a unit charge from b to a; mathematically

$$v_{ab} \triangleq \frac{dw}{dq} \tag{3.4}$$

Voltage-or electromotive force (emf) / electric potential difference-is the energetic driving force that moves charge carriers through a circuit pathway. Voltage is the work required to move a unit charge through a circuit element between two specific coordinate points, the unit Volt (V) is defined as 1 Joule per coulomb (J/C).

[FL*]

- **Power and Energy**

[FL*] Electric power is the time rate at which work is performed or energy is converted within a circuit element. The power formula is derived using the chain rule from calculus

$$P = \frac{dW}{dt} = \left(\frac{dW}{dQ}\right)\left(\frac{dQ}{dt}\right) = VI \tag{3.5}$$

It is the product of the potential difference across an element and the current passing through it, the unit of power is Watt (W), defined as 1 Joule per second (J/s).

[FL*] Energy is the total capacity to perform work over an operational time domain. The formula is

$$W = \int_0^t P(\tau)d\tau \tag{3.6}$$

represented as the time integral of electrical power over a specified period, the unit is Joule (J), though commercial utilities use the Kilowatt-hour (kWh).
The energy formula for a constant DC power profile is

$$W = Pt = Vit \tag{3.7}$$

Theorem 3.1: Ohm's Law

Ohm's Law states that for an ideal conductor at a constant temperature, the current flowing through it is directly proportional to the potential difference across it. The formula is

$$V = IR \tag{3.8}$$

where I is the current and R is the electrical resistance measured in Ohms (Ω).

Theorem 3.2: Kirchoff's Current Law (KCL)

Kirchoff's Current Law is based on the law of conservation of charge. It states that charge cannot be created or destroyed at any node (juntion) within a circuit. That means the algebraic sum of all currents entering and leaving a node must equal zero.

$$\sum_{k=1}^{n} I_k = 0 \tag{3.9}$$

Theorem 3.3: Kirchoff's Voltage Law (KVL)

Kirchoff's Voltage Law is based on the law of conservation of charge. It states that the net energy gained by a unit charge moving around a closed path must equal the net energy lost.

$$\sum_{k=1}^{n} V_k = 0 \tag{3.10}$$

The algebraic sum of all electrical potential differences (voltages) around any closed loop must equal zero.

Definition 3.1: Passive Sign Convention

To properly balance power calculations across network equations, circuits use the Passive Sign Convention:

- If conventional current enters the positive terminal of a component, the power is positive ($P > 0$), meaning the element absorbs (consumes) energy.
- If conventional current leaves the positive terminal (enters the negative terminal), the power is negative ($P < 0$), meaning the element supplies energy.

By using Tellegen's Theorem that states : In any lumped electrical network, the sum of delivered power equals the sum of absorbed power, thus

$$\sum P_{supplied} = \sum P_{absorbed} \tag{3.11}$$

$$\sum_{k=1}^{N} V_k I_k = 0 \tag{3.12}$$

- **DC Circuit Elements**

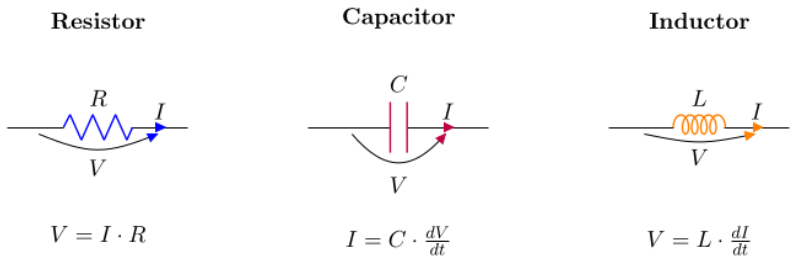


Figure 3.2: *Schematic layout for core passive elements: resistors, capacitors, and inductors under standard active polarities.*

[FL*]
[FL*]

- **Applications**

[FL*]
[FL*]

- **Problem Solving**

[FL*]
[FL*]

II. AC Circuits

i. Sinusoids and Phasors

- **General Theory of nth Order Linear Equations**

[FL*] The first
[FL*]

- **The Method of Variation of Parameters**

[FL*] The first
[FL*]

III. Pulse Circuits

IV. Advanced Circuit Analysis

i. Introduction to the Laplace Transform

- **The Occurrence of Two-Point Boundary Value Problems**

[FL*] The first
[FL*]

- **Nonhomogeneous Boundary Value Problems**

[FL*] The first
[FL*]

Chapter 4

Digital

*"The only thing worse than being talked about is not being talked about." - **Oscar Wilde***

IF

Chapter 5

Hardware

"It is known that man has five Exterior Senses, and every sense is ignorant of each other; for the Nose knows not what the Eyes see, nor the Eyes know what the Ears hear, neither do the Ears know what the Tongue tastes...." - ***Cavendish 2***

IF

Chapter 6

Information Systems and Science

*"It is from their foes, not their friends, that cities learn the lesson of building high walls." - **Aristophanes***

IF

Chapter 7

Physical Technology and Science

*"One doesn't discover new lands without losing sight of the shore." - **Andre Gide***

IF

Chapter 8

Bio-Electronics and Bio-Imaging

"It is not the strongest of the species that survive, but the one most responsive to change." - ***Charles Darwin***

IF

Chapter 9

Energy and Environment

"Nature herself has imprinted on the minds of all the idea of God." - ***Cicero***

IF

Chapter 10

Reference and Bibliography

Bibliography

[1] Colley, Susan Jane (2012) Vector Calculus 4th Edition, Pearson, Boston, MA, United States.

[2] Sadiku, Matthew, Alexander, Charles K. (2013) Fundamental of Electric Circuits 6th Edition, McGraw-Hill, New York, NY, United States.

[3] Anton, Rorres (2006) Elementary Linear Algebra with Supplemental Applications 10th Edition, Wiley, Boston, United States.

[4] Anton, Howard, Rorres, Chris, Elementary Linear Algebra with Supplemental Applications 12th edition, Wiley, Hoboken, New Jersey, United States.

[5] Dorf, Richard C., Bishop, Robert H. (2010) Modern Control Systems 12th Edition, Pearson, New Jersey, United States.